CertyIQ

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

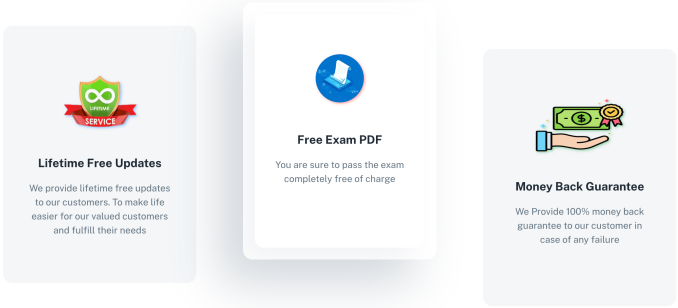
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Microsoft

(AB-620)

Designing and Building Integrated AI Agent Solutions in Copilot Studio

Total: **91 Questions**

Link: https://certyiq.com/papers/microsoft/ab-620

Question: 1 CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:

Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to configure the agent in Copilot Studio to meet Blue Vender's Responsible AI and data protection requirements.
Which three actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

Responsible AI and data protection requirements	Answer Area	
Add custom prompt modifications.	1	
Conduct adversarial testing.	2	
Define Data Loss Prevention (DLP) policies.	3	
Integrate with Microsoft Purview.		
Disable generative answers.		
Configure content moderation filters.		

Answer:

Responsible AI and data protection requirements	Answer Area	
Add custom prompt modifications.	1	Add custom prompt modifications.
Conduct adversarial testing.	2	Configure content moderation filters.
Define Data Loss Prevention (DLP) policies.	3	Conduct adversarial testing.
Integrate with Microsoft Purview.		
Disable generative answers.		
Configure content moderation filters.		

Explanation:

Add custom prompt modifications

Why it's first: This is the base layer of safety. Custom instructions (or system prompts) establish the fundamental boundaries, personality, and contextual rules for the generative AI model inside Microsoft Copilot Studio.

It explicitly tells the LLM how to behave, what tone to maintain, and what topics or data sources are strictly off-limits before any user interaction happens.

Configure content moderation filters

Why it's second: Once the core instructions are set, you apply real-time filtering layers. Content moderation filters act as an active validation shield that monitors incoming user prompts and outgoing model completions.

It detects and blocks toxic, unsafe, or off-policy phrasing that a custom prompt alone might fail to stop, completely satisfying the requirement that "filters must be used to enforce tone, safety, and compliance."

Conduct adversarial testing

Why it's last: Testing always occurs after all defensive measures and configuration changes have been implemented.

Adversarial testing (commonly known as red-teaming) attempts to deliberately trick, bypass, or prompt-inject the copilot. Performing this as the final step verifies that your prompt modifications and content moderation filters actually function as intended under realistic stress scenarios before deployment.

Why the other choices are incorrect

Define Data Loss Prevention (DLP) policies: DLP policies control environment-level connectors and data flow at the Power Platform administration tier, rather than controlling the specific conversational tone, safety, and compliance filters inside a generative copilot.
Integrate with Microsoft Purview: While Purview is excellent for enterprise compliance and data lifecycle auditing, the specific requirement explicitly emphasizes adjusting prompt instructions and filters directly within the copilot architecture.
Disable generative answers: The case study specifically requires using generative answers safely ("Generative responses must be constrained..."), so disabling them completely defeats the fundamental purpose of the requested AI deployment.

Question: 2 CertyIQ

A company uses an agent in Copilot Studio that produces structured responses using a reusable instruction template. The responses follow the required tone and format. During a compliance review, the team discovers that the responses were generated using a model that is not on the approved list.
The company requires that the reusable instruction template continues to be used and the model used for prompt-driven responses meets governance requirements.
The solution must ensure that prompt-driven responses are generated using an approved model.
You need to configure the solution that meets the requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Select an approved model from the Microsoft Foundry model catalog in the configuration that is used for prompt-driven response generation.

B. Configure Azure AI Search as the grounding source so the approved model requirement is enforced during response generation.

C. Ensure the reusable instruction template is the one applied to the agent that generates the structured response.

D. Publish the agent so that the model used by prompt-driven responses is updated automatically.

E. Enable citations so generated responses can be traced and comply with governance requirements.

F. Add a line to the instruction template that requires the agent to use only approved models when generating responses.

Answer: AC

Question: 3 CertyIQ

DRAG DROP
A company needs to ground an agent in Copilot Studio answers using live enterprise data from a supported system via a Microsoft Power Platform connector. To meet the business needs, the builder must meet the following requirements:
• Authenticate using a valid connection.
• Add the connector as a real-time knowledge source.
• Select the tables for knowledge grounding.
You need to add a Power Platform connector as a real-time knowledge source for the agent.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

REST API	Answer Area	
Select the tables to include as the knowledge source.	1	Add a knowledge source.
Add the connector to the agent.	2	Select a real-time connector from the Add knowledge dialog.
Select a real-time connector from the Add knowledge dialog.	3	Sign in and create a new connection or reuse an existing connection.
Sign in and create a new connection or reuse an existing connection.	4	Select the tables to include as the knowledge source.
Add a knowledge source.	5	Add the connector to the agent.

Answer:

1. Create a connection using a Microsoft Power Platform connector.2. In the Copilot Studio agent, navigate to the Knowledge tab.3. Select Add a knowledge source and choose the Power Platform connector.4. Select the specific tables to ground the agent's answers.

Question: 4 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
An agent uses a flow that calls an external service which can occasionally fail or time out.
When a failure occurs, the agent must meet the following requirements:
• Must not terminate silently.
• Must send a notification containing the error details.
You need to configure the agent flow so that failures are handled in a controlled and predictable way.
Solution: Terminate the flow immediately on error.
Does the solution meet the goal?

A. Yes
B. No

Exception Handling in Agent Flows (AB-620T00 / Power Automate Error Handling Patterns): Under Microsoft's standard error-handling architecture for flows invoked by Copilot Studio agents (the Try-Catch-Finally pattern using Scope actions and Configure run after), simply terminating a flow immediately upon failure causes the flow to abort abruptly without executing downstream alerting logic.

Answer: B

Question: 5 CertyIQ

A company requires an agent in Copilot Studio to answer questions using content stored in a third-party knowledge base system and reuse that content for knowledge grounding.
The agent must ground answers using indexed enterprise knowledge from a non-Microsoft system.
You need to choose the correct integration approach.
What should you use?

A. Call the third-party API from a topic for each question.
B. Add a Copilot Studio tool as a real-time knowledge source.
C. Add a Copilot Studio tool and call it from a topic.

D. Add a Copilot connector knowledge source for authorized content.

Answer: D

Indexed External Knowledge via Copilot Connectors (AB-620T00 / AB-100 — Add knowledge sources to an agent): In Microsoft Copilot Studio, enterprise knowledge grounding falls into two primary integration patterns for external systems:
Microsoft 365 Copilot connectors (formerly Microsoft Graph connectors): Ingest external data from non-Microsoft systems (such as Confluence, ServiceNow Knowledge, Jira, Salesforce, or custom external repositories) directly into Microsoft Graph, creating a searchable semantic index with item-level permission trimming (authorized content). Once indexed, Copilot Studio agents natively consume the Copilot connector as a reusable Knowledge source.
Real-time Power Platform connectors: Query live transactional data at runtime without indexing it into Microsoft Graph (used for live records rather than indexed knowledge bases).

Question: 6

CertyIQ

DRAG DROP
A company uses a supported Power Platform connector to provide real-time access to enterprise records for knowledge grounding in an agent in Copilot Studio.
To meet the business needs, the knowledge source must provide the following functionality:
• Avoid copying enterprise data into an index.
• Keep source-system access controls enforced.
• Access data by calling the system at runtime.
You need to configure the connector knowledge source.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Use real-time connector.
- Use existing permissions.
- Retrieve data without indexing.
- Add Azure AI Search as a knowledge source.

Knowledge source

Requirement	Configuration
Avoid copying enterprise data into an index.	
Keep source-system access controls enforced.	
Access data by calling the system at runtime.	

Answer:

To meet the requirements for real-time access to enterprise records in Copilot Studio while maintaining data sovereignty and security:* Avoid copying enterprise data into an index: Configure Search endpoint to point to the connector/API rather than an indexed data store.* Keep source-system access controls enforced: Use Connector authentication (or User-delegated authentication) to ensure the system respects the user's specific permissions.* Access data by calling the system at runtime: Configure Action-based knowledge (which utilizes Power Platform connectors to fetch data dynamically at runtime).Answer:* Avoid copying enterprise data into an index: Connector* Keep source-system access controls enforced: Connector authentication* Access data by calling the system at runtime: Action-based knowledge

Question: 7

CertyIQ

A tenant administrator configures a Copilot connector that indexes content from an external knowledge system into Microsoft Graph.
A builder is building an agent that must:
• Use the indexed enterprise knowledge during answer generation.
• Avoid duplicating or manually uploading the same content.
• Avoid invoking the system as a transactional tool.
You need to configure the agent so that responses are grounded in the authorized knowledge sources.
What should you do?

A. Add the configured enterprise knowledge source to the agent's knowledge.
B. Create a new custom connector inside the agent configuration.
C. Add the connector as a tool and invoke it from a topic.
D. Upload documents from the external system directly into the agent's file knowledge store.

Answer: A

Question: 8

CertyIQ

DRAG DROP
A solution containing an agent in Copilot Studio is deployed to multiple environments that require different configuration values.
Configuration values must vary by environment without modifying the solution contents.
Configuration values must meet the following requirements:
• Allow values to be set during import.
• Exclude values from the agent definition.
• Run a dedicated agent instance for each individual environment.
You need to associate each requirement with the configuration approach.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Use channel-specific settings.
- Define environment variables in the solution.
- Publish distinct agent versions for each environment.
- Hardcode the configuration values in the agent configuration.

ALM-related configurations

Requirement	Configuration
Allow values to be set during import.	Define environment variables in the solution.
Exclude values from the agent definition.	Define environment variables in the solution.
Run a dedicated agent instance for each individual environment.	Publish distinct agent versions for each environment.

Answer:

* Allow values to be set during import: Environment variables* Exclude values from the agent definition: Environment variables* Run a dedicated agent instance for each individual environment: Environment-specific configuration

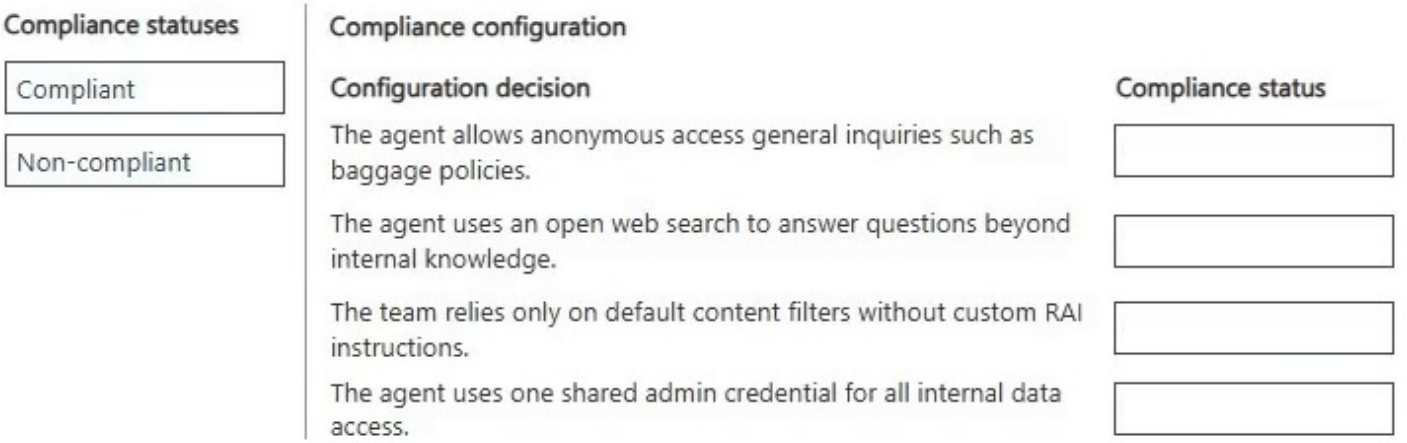
Question: 9

CertyIQ

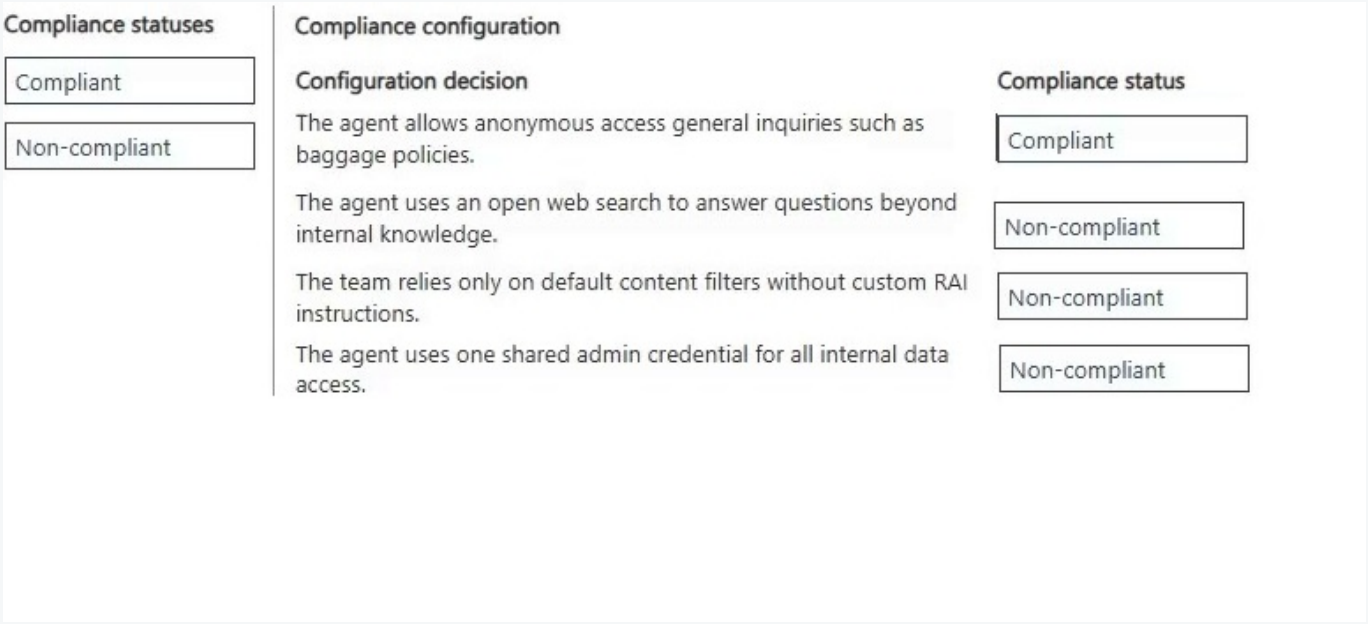
DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.

The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.

Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to evaluate whether the current configuration decisions for the Blue Yonder Copilot agent comply with the company's security and governance policies.
Which compliance status should you assign to each configuration decision? To answer, move the appropriate compliance statuses to the correct configuration decisions. You may use each compliance status once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

(Compliant): Baggage policies are public informational data. Allowing unauthenticated (anonymous) access for standard customer service FAQs is completely secure and follows standard architectural practices.

(Non-compliant): Utilizing ungrounded open web searches introduces severe risks of hallucinations and off-policy responses. This directly violates the case study's core mandate that generative answers must be securely constrained.

(Non-compliant): As established in the safety deployment requirements, custom system instructions (RAI prompts) are mandatory to enforce the specific tone, safety compliance, and operational boundaries of the company. Relying solely on default settings is insufficient.

(Non-compliant): Employing a single shared administrator credential for all internal backend access breaches basic Zero Trust security guidelines and data privacy principles (Least Privilege). It allows users to potentially bypass data access controls or view internal data they are not authorized to see.

Question: 10 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -

Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to deploy the Blue Yonder Copilot agent to the public website and Microsoft Teams while ensuring compliance with the company's security and Responsible AI requirements.
Which two actions should you perform before making the agent available on both channels? Each correct answer presents part of the solution.

NOTE: Each correct selection is worth one point.

A. Configure prompt modifications to enforce tone, disclaimers, and refusal behavior at the system level.
B. Manually add disclaimers to each topic before publishing.
C. Embed the web channel and then rely on channel-level settings to enforce content moderation.
D. Configure Power Platform DLP policies to restrict unauthorized data connectors.
E. Publish the agent and then enable Responsible AI filters individually for each channel.

Answer: AD

Explanation:

A. Configure prompt modifications to enforce tone, disclaimers, and refusal behavior at the system level.

D. Configure Power Platform DLP policies to restrict unauthorized data connectors.

Action A (Responsible AI Compliance): Relying on system-level prompt modifications ensures that regardless of the topic or channel (Public Web or Microsoft Teams), the generative AI model inherently follows enterprise guardrails regarding tone, required disclaimers, and refusal bounds.
Action D (Security Compliance): Power Platform Data Loss Prevention (DLP) policies act as the security firewall. They control and block unauthorized connectors at the environment tier, preventing the copilot from accidentally leaking or retrieving data through non-compliant external channels.

Why the other options are incorrect

B is an operational nightmare; manually updating individual topics fails to secure globally generated AI answers.
C & E are architectural misconceptions; content moderation and Responsible AI filters in Microsoft Copilot Studio are centrally managed at the agent/system tier before publication, rather than handled piece-by-piece on distinct channels.

Question: 11 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -

Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -

Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to configure the Blue Yonder Copilot agent's responses in accordance with the company's content control and platform requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Add required disclaimer text inside each individual topic.
B. Configure prompt instructions that include disclaimer text.
C. Use Markdown syntax within response content.
D. Insert HTML formatting directly into topic responses.
E. Duplicate disclaimer text across reusable topics.

Answer: BC

Explanation:

B. Configure prompt instructions that include disclaimer text.

C. Use Markdown syntax within response content.

Action B (Content Control Compliance):

Adding required disclaimer instructions at the system prompt level ensures that the underlying generative model consistently appends or enforces the necessary legal and safety compliance guidelines globally, preventing the need to manually update static topics.

Action C (Platform Requirement Compliance):

Microsoft Copilot Studio natively supports and recommends **Markdown** syntax within response message nodes. It ensures content renders consistently (such as bolding, lists, and hyperlinks) across both the public website channel and Microsoft Teams.

Question: 12

CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.

Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to configure the agent in Copilot Studio to use internal and external partner knowledge sources to answer user questions about the airline services.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Use a Microsoft Graph connector to index the partner's travel advisory content.

B. Write individual Q&A pairs for each document as separate topics.

C. Enable unrestricted web search for the agent.

D. Add the internal policy documents as a knowledge source.

Answer: AD

Explanation:

A. Use a Microsoft Graph connector to index the partner's travel advisory content.

For structured or third-party partner data (like external travel advisory sites or repositories), a **Microsoft Graph connector** is the ideal modern solution. It indexes the external data directly into the enterprise search

index, allowing Copilot Studio to safely ingest and query partner information without exposing it to the open web.

D. Add the internal policy documents as a knowledge source.

Adding the internal policy documents directly as native knowledge sources in Copilot Studio allows the agent to immediately use Retrieval-Augmented Generation (RAG) to ground answers based only on approved, private company content.

Why the Other Options are Incorrect

B. Write individual Q&A pairs for each document: Manually writing hardcoded Q&A pairs for entire sets of documents is highly inefficient, does not scale, and defeats the purpose of using Copilot's automated, generative AI retrieval features.
C. Enable unrestricted web search: Turning on a broad, unrestricted web search will crawl public search engine data (like Bing). This can lead to unverified or incorrect generic answers rather than pulling exclusively from the verified external partner knowledge you need.

Question: 13 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).

Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.

Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to ensure that every AI-generated response from the agent in Copilot Studio includes a disclaimer that complies with the company's security and governance policies.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Add a greeting message that includes the disclaimer.
B. Add a prompt modification in the generative answers node settings.
C. Create a disclaimer topic that always runs first.
D. Disable generative answers and use only pre-authored responses.
E. Edit every topic's first message to include the disclaimer.

Answer: BC

Explanation:

B. Add a prompt modification in the generative answers node settings.
C. Create a disclaimer topic that always runs first.

Why These Actions are Correct

1. **Add a prompt modification in the generative answers node settings (Option B):**
2. By modifying the custom instructions (prompt engineering) within the **Generative Answers node**, you can explicitly instruct the large language model (LLM) to automatically append the required governance and security disclaimer text to the end of every answer it dynamically synthesizes from your data sources.
3. **Create a disclaimer topic that always runs first (Option C):**
4. Creating a dedicated topic with an orchestration trigger (such as the system trigger **"An AI-generated response is about to be sent"**) acts as a universal interception layer. This ensures that regardless of which topic or knowledge base was queried, the disclaimer logic evaluates and appends the warning message uniformly before sending the payload out to the user.

Why the Other Options are Incorrect

A. Add a greeting message: A greeting message is only shown once at the absolute beginning of a user session. It does not fulfill the compliance mandate that requires it on every individual AI-generated response.
D. Disable generative answers: While this ensures no unapproved answers are sent, it completely removes the conversational AI capability from the agent, which violates the premise of handling AI-generated responses.
E. Edit every topic's first message: This is an administrative nightmare that is highly prone to human error, does not scale, and only covers hardcoded pre-authored messages rather than dynamically generated conversational responses.

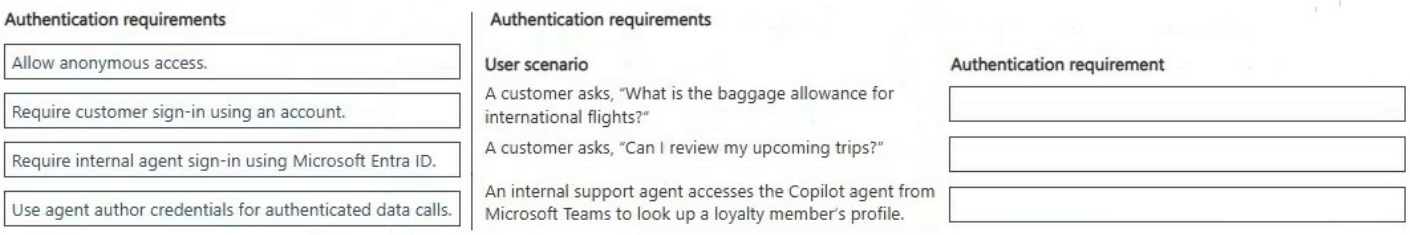
Question: 14

CertyIQ

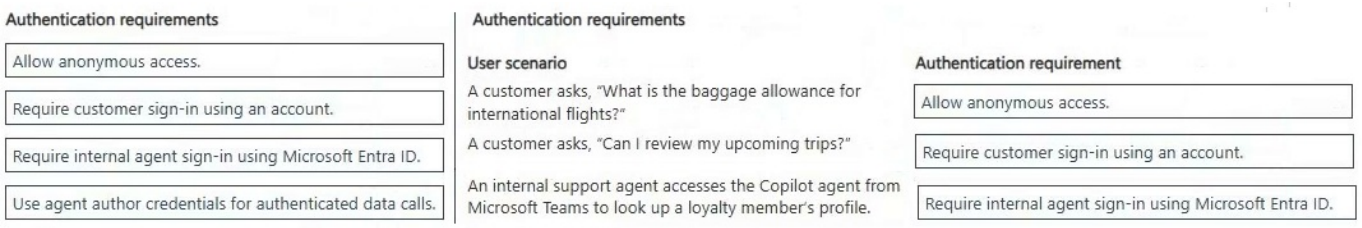
DRAG DROP -

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.

Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to determine which authentication model should be applied to each Blue Yonder Copilot interaction scenario to comply with the company's security and governance requirements.
Which authentication requirement should you apply to each scenario? To answer, move the appropriate authentication requirements to the correct user scenarios. You may use each authentication requirement once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Baggage allowance is general, non-sensitive public information. Customers should be able to view these details directly on a public website without creating an account or logging in, making **Allow anonymous access** the appropriate choice.

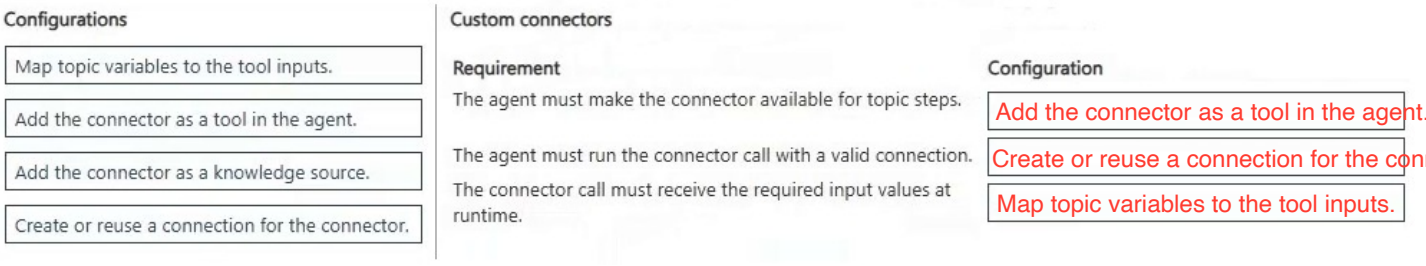
Reviewing personal upcoming trips involves accessing Personally Identifiable Information (PII) and private booking records. To ensure data privacy, the copilot must securely identify who the user is by forcing a **customer sign-in using an account**.

Since an internal customer support representative is accessing the system within the company's internal tenant (Microsoft Teams) to view sensitive loyalty customer profiles, they must authenticate securely via the enterprise identity provider system, which is **Microsoft Entra ID**.

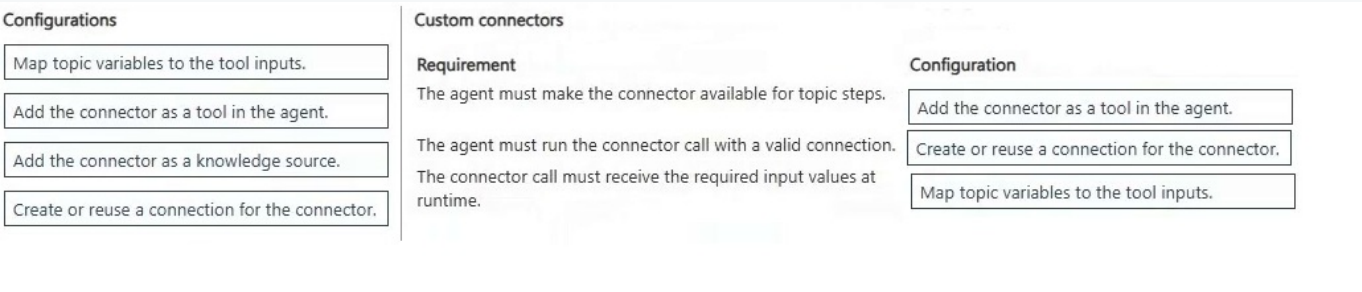
Question: 15

CertyIQ

DRAG DROP -
A company has an existing custom connector that is approved and available in the environment. A builder wants an agent in Copilot Studio to call the connector during a conversation to retrieve information from an internal system.
To meet the business needs, the solution must meet the following requirements:
The agent must make the connector available for topic steps.
The agent must run the connector call with a valid connection.
The connector call must receive the required input values at runtime.
You need to configure the agent so the custom connector can be used as a tool.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

In Microsoft Copilot Studio, actions or connectors must be registered or enabled within the bot's plugin architecture to become visible. Choosing **Add the connector as a tool in the agent** allows authors to select and execute the custom connector inside regular topic authoring canvas steps.

Connectors are definitions of APIs, but they cannot authenticate or talk to a backend without a concrete security context. You must **Create or reuse a connection for the connector** to establish valid authentication credentials (like OAuth or API keys) so the engine can execute the request successfully.

For a custom connector tool to dynamically fetch accurate data (such as searching for a flight using a specific booking ID), data collected from the user during the chat session must pass into the API request. You **Map topic variables to the tool inputs** to satisfy this requirement.

Question: 16 CertyIQ

DRAG DROP -
A company needs to ground an agent in Copilot Studio answers using live enterprise data from a supported system via a Microsoft Power Platform connector.
To meet the business needs, the builder must meet the following requirements:
Authenticate using a valid connection.
Add the connector as a real-time knowledge source.
Select the tables for knowledge grounding.
You need to add a Power Platform connector as a real-time knowledge source for the agent.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

REST API
Select Sign in and create a new connection or reuse an existing connection.
Select the tables to include as the knowledge source.
Select Add knowledge from the agent Knowledge experience.
Select the real-time connector from the Add knowledge dialog.
Select Add to agent to complete the connection.

Answer Area	
1	Add knowledge from the agent Knowledge experience.
2	Select the real-time connector from the Add knowledge dialog.
3	Select Sign in and create a new connection or reuse an existing connection.
4	Select the tables to include as the knowledge source.
5	Select Add to agent to complete the connection.

Answer:

REST API	Answer Area	
Select Sign in and create a new connection or reuse an existing connection.	1	Select Add knowledge from the agent Knowledge experience.
Select the tables to include as the knowledge source.	2	Select the real-time connector from the Add knowledge dialog.
Select Add knowledge from the agent Knowledge experience.	3	Select Sign in and create a new connection or reuse an existing connection.
Select the real-time connector from the Add knowledge dialog.	4	Select the tables to include as the knowledge source.
Select Add to agent to complete the connection.	5	Select Add to agent to complete the connection.

Explanation:

Select Add knowledge from the agent Knowledge experience.

Start from the agent configuration area. The **Knowledge** experience is where you add external data sources.

Select the real-time connector from the Add knowledge dialog.

In the Add knowledge dialog, choose the appropriate connector type. For REST API data, select the **real-time connector** option so the agent can retrieve information dynamically rather than storing a static copy.

Select Sign in and create a new connection or reuse an existing connection.

The connector requires authentication. You either create a new connection by providing credentials/details or select an existing connection that already has the required authentication settings.

Select the tables to include as the knowledge source.

After the connection is established, the available data objects/tables exposed by the connector are displayed. Choose which tables or data entities should be available to the agent as knowledge.

Select Add to agent to complete the connection.

Confirm the selection and attach the configured knowledge source to the agent. The agent can now use the connected REST API data when responding to user queries.

Question: 17 CertyIQ

HOTSPOT -
You deploy an agent in Microsoft Copilot Studio that triggers a cloud flow to update customer records in Microsoft Dataverse.
You observe the following after deploying the agent:
Several runs fail due to a temporary authentication issue.
The flow continues to trigger and generate additional failed runs.
You must:
Stop additional failed executions while troubleshooting.
Process a failed execution after resolving the authentication issue.
Validate if the most recent executions succeeded immediately after resolution.
You need to use the appropriate monitoring and management actions to restore normal flow operation and validate successful execution.
Which action should you perform for each requirement? To answer, select the appropriate options in the answer

area.
NOTE: Each correct selection is worth one point.

Flow monitoring actions

Requirement	Action
Stop additional failed executions while troubleshooting.	Delete the flow. Disable the flow. Cancel the most recent run. Modify the trigger condition.
Process a failed execution after resolving the authentication issue.	Restart the flow. Refresh the flow. Resubmit the run. Recreate the trigger.
Validate if the most recent executions succeeded immediately after resolution.	Republish the flow. Review the Activity history. Export the flow to a solution. Inspect the Analytics summary.

Answer:

Flow monitoring actions

Requirement	Action
Stop additional failed executions while troubleshooting.	Delete the flow. Disable the flow. Cancel the most recent run. Modify the trigger condition.
Process a failed execution after resolving the authentication issue.	Restart the flow. Refresh the flow. Resubmit the run. Recreate the trigger.
Validate if the most recent executions succeeded immediately after resolution.	Republish the flow. Review the Activity history. Export the flow to a solution. Inspect the Analytics summary.

Explanation:

Disable the flow.

Disabling the flow prevents the trigger from firing and stops any new runs from being created while you investigate the issue. Deleting the flow is permanent, canceling only affects one run, and modifying the trigger condition is unrelated to temporarily stopping executions.

Resubmit the run.

After fixing the authentication problem (for example, updating the connection), use **Resubmit the run** to execute the failed flow instance again. This retries the same inputs without waiting for a new trigger event.

Review the Activity history.

The Activity history (Run history) displays the latest flow executions, including their status (Succeeded, Failed, Running), timestamps, and error details. This is the quickest way to verify whether the issue has been resolved. Analytics provides aggregated trends rather than immediate run-by-run validation.

Question: 18 CertyIQ

DRAG DROP -
A company is preparing an agent flow so that it can be invoked by an agent during conversations.
The agent flow must meet the following requirements:
The agent must be able to trigger the flow.
The agent flow must be verified.
You need to prepare an agent flow so that it can be used by the agent.
In which order should you perform the actions to prepare the agent flow? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

Agent flow readiness steps
Configure the invocation trigger that allows the agent to call the agent flow.
Make the agent flow available for agent usage.
Validate the agent flow behavior with representative test runs.
Configure the actions that implement the required behavior.
Create a new agent flow artifact in the environment.

Answer Area	
1	Create a new agent flow artifact in the environment.
2	Configure the invocation trigger that allows the agent to call the agent flow.
3	Configure the actions that implement the required behavior.
4	Validate the agent flow behavior with representative test runs.
5	Make the agent flow available for agent usage.

Answer:

Agent flow readiness steps
Configure the invocation trigger that allows the agent to call the agent flow.
Make the agent flow available for agent usage.
Validate the agent flow behavior with representative test runs.
Configure the actions that implement the required behavior.
Create a new agent flow artifact in the environment.

Answer Area	
1	Create a new agent flow artifact in the environment.
2	Configure the invocation trigger that allows the agent to call the agent flow.
3	Configure the actions that implement the required behavior.
4	Validate the agent flow behavior with representative test runs.
5	Make the agent flow available for agent usage.

Explanation:

1. **Create the artifact**: You begin inside your Power Platform environment by creating a new automated or instant cloud flow package/artifact designated to handle your specialized automation logic.
2. **Configure the invocation trigger**: Next, you define the entry point — typically using the **"Run a flow from Copilot"** trigger node — where you declare exactly what input variables, types, and schemas the agent must provide when invoking the flow.
3. **Configure the actions**: With the input parameters established, you construct the internal business logic by adding connector blocks, data operations, variables, or API calls that execute the intended process.
4. **Validate behavior**: Before publishing, you perform runtime tests or debug cycles within the flow builder canvas using mock or representative parameters to verify that the steps execute smoothly and output data correctly.
5. **Make available for usage**: Once verified, you save, turn on, or share the flow so it can be successfully discovered, mapped, and executed inside **Copilot Studio** as an operational tool.

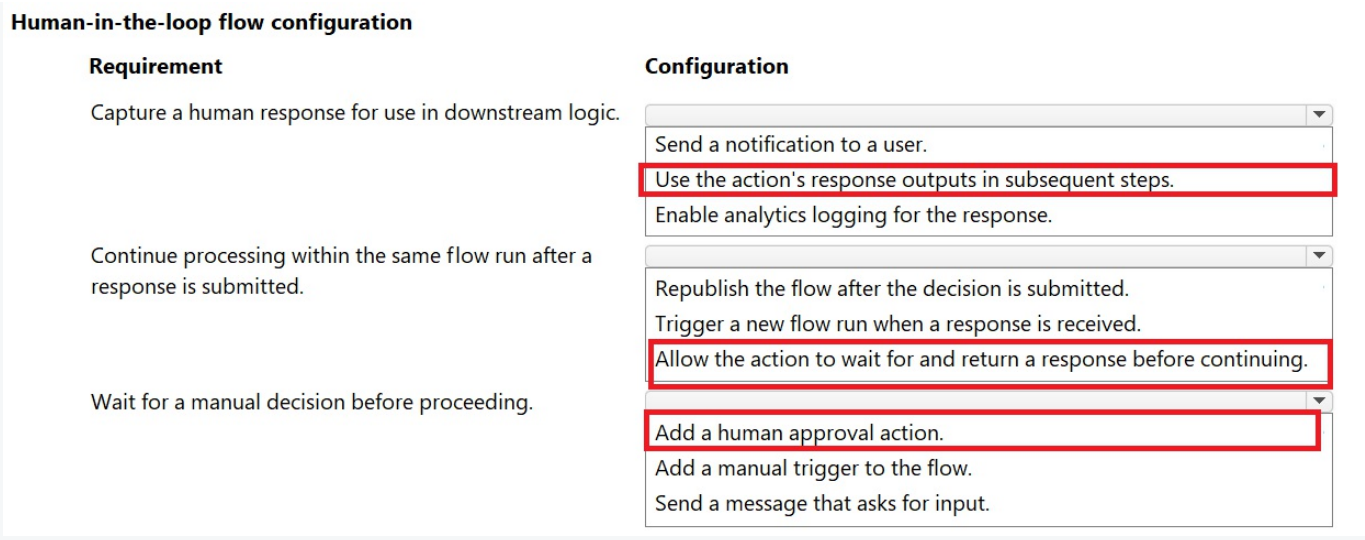
Question: 19 CertyIQ

HOTSPOT -
A company uses an agent flow that occasionally requires human input before continuing execution.
Some automated actions must pause until a human provides a decision or additional information. The flow must be configured to:
Capture a human response for use in later steps.
Continue processing within the same flow run after the response is submitted.
Wait for a manual decision before proceeding.
You need to configure a human-in-the-loop agent flow.
Which setting should you configure for each requirement? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Human-in-the-loop flow configuration

Requirement	Configuration
Capture a human response for use in downstream logic.	Send a notification to a user. **Use the action's response outputs in subsequent steps.** Enable analytics logging for the response.
Continue processing within the same flow run after a response is submitted.	Republish the flow after the decision is submitted. Trigger a new flow run when a response is received. **Allow the action to wait for and return a response before continuing.**
Wait for a manual decision before proceeding.	**Add a human approval action.** Add a manual trigger to the flow. Send a message that asks for input.

Answer: Human-in-the-Loop Orchestration in Agent Flows (AB-620T00 — Create and manage agent flows in Microsoft Copilot Studio): When an autonomous or conversational agent workflow requires human oversight or sign-off mid-execution, Microsoft Copilot Studio and Power Automate implement Human-in-the-Loop (HITL) patterns using native Approvals actions (such as Start and wait for an approval or Post adaptive card and wait for a response):



Explanation:

1. Use the action's response outputs in subsequent steps.

Why it fits: "Downstream logic" in an automated cloud workflow refers to any action cards or condition evaluations that execute later in the sequence. To pass a choice made by a person (such as an approval outcome or typed remarks) further down the line, you must reference the dynamic outputs produced by that specific approval or response action.

2. Allow the action to wait for and return a response before continuing.

Why it fits: By default, asynchronous approval actions inside Power Automate pause the execution thread of that specific, active flow run. By enabling the step to **wait for and return a response**, the exact same workflow instance goes into a dormant state and safely wakes back up to finish its remaining actions the moment the reviewer submits their decision.

3. Add a human approval action.

Why it fits: The Power Automate **Approvals** connector is engineered specifically to implement human-in-the-loop validation checkpoints. It generates a formal, auditable task envelope via Teams, Email, or the Power Automate portal, forcing the cloud flow execution engine to halt completely until an authorized user actively clicks a decision button.

Question: 20 CertyIQ

HOTSPOT -
A company uses an agent that invokes an agent flow to exchange information during a conversation.
The company requires that the agent send data into the flow and receive structured results back from the same flow run. To support this business need, the flow must be configured to do the following:
Capture the data provided by the agent.
Return data results to the agent.
You need to configure the flow so that it can exchange data with the agent.
What should you configure for each requirement? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Agent flow input and output configuration

Requirement	Configuration
Capture the data provided by the agent.	Use an internal variable. **Add a text input parameter.** Store values only in the topic.
Return data results to the agent.	Log values to run history. Store values only in the flow. **Add a text output parameter.**

Answer:

Agent flow input and output configuration

Requirement	Configuration
Capture the data provided by the agent.	Use an internal variable. **Add a text input parameter.** Store values only in the topic.
Return data results to the agent.	Log values to run history. Store values only in the flow. **Add a text output parameter.**

Explanation:

1. Add a text input parameter.

Why it fits: When an agent (Copilot) invokes an automated cloud flow to offload processing logic, it needs a way to hand off variables collected from the user conversation (such as names, account numbers, or text commands) to the flow. Configuring an explicit **input parameter** directly on the flow's invocation trigger establishes the required inbound interface slots to accept this incoming data.

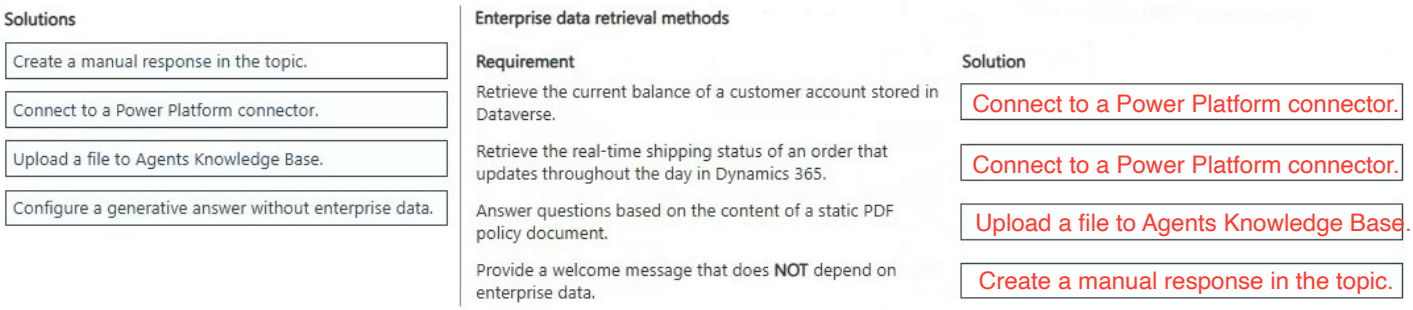
2. Add a text output parameter.

Why it fits: Once the automated flow concludes its backend processing, data operations, or API lookups, it must send the finalized results back into the conversation context. Defining a **text output parameter** on the flow's termination response block creates the outbound interface payload that the agent can read and reuse in its subsequent conversation topics.

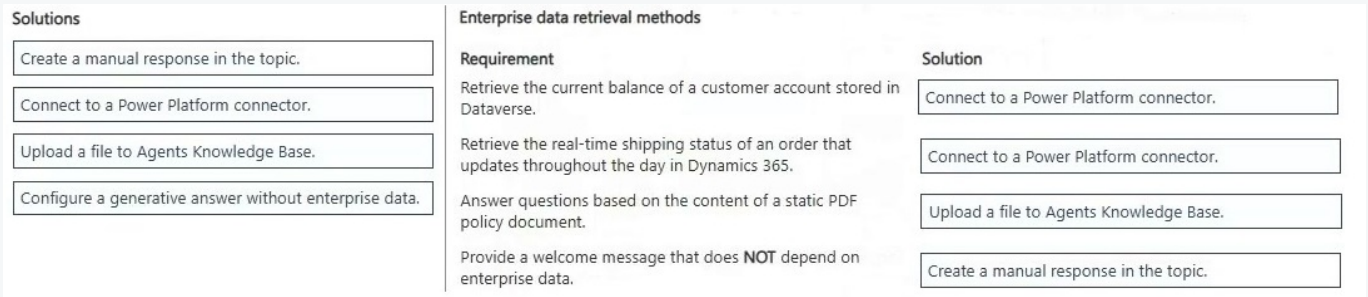
Question: 21

CertyIQ

DRAG DROP -
You are configuring an agent in Copilot Studio for an organization. The organization uses Microsoft Dataverse, Dynamics 365, and SharePoint Online.
The agent must retrieve information from enterprise systems and internal documents. The agent must meet the following requirements:
Must retrieve the current balance of a customer account stored in Dataverse.
Must retrieve the real-time shipping status of an order that updates throughout the day in Dynamics 365.
Must answer questions based on the content of a static PDF policy document.
Must provide a welcome message that does NOT depend on enterprise data.
You need to configure the agent to meet the requirements.
How should the agent retrieve the data? To answer, move the appropriate solutions to the correct requirements. You may use each solution once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1 & 2. Connect to a Power Platform connector.

Why it fits: Both Microsoft Dataverse and Dynamics 365 are transactional enterprise data stores. When an agent needs to pull precise, transactional variables (like an account balance) or track highly transient information that changes continuously throughout the day (like a shipping status), it cannot rely on scheduled search indexing. Instead, it must utilize a **Power Platform connector** to run a direct, real-time query against the source system at that exact moment in the conversation.

3. Upload a file to Agents Knowledge Base.

Why it fits: Static documents (such as employee handbooks, corporate guidelines, or PDF policy manuals) contain unstructured informational text that doesn't change frequently. Uploading the PDF directly into the agent's **Knowledge Base** allows the built-in system to parse, chunk, and index the file so the foundational AI model can reference it for Retrieval-Augmented Generation (RAG).

4. Create a manual response in the topic.

Why it fits: Simple conversational steps like greetings, introductions, or fixed welcome messages do not

require external databases, AI search crawls, or live data logic. Standardizing this greeting as a **manual message node** within a topic ensures a quick, uniform user experience completely independent of enterprise data pipelines.

Question: 22 CertyIQ

An agent calls a flow. The agent requires structured output values to be returned.
The agent receives unexpected or empty values.
You need to configure the agent so that data is exchanged correctly between the agent and the flow.
What should you do?

A. Validate parameter definitions.
B. Change the trigger schema.
C. Review the flow run history.
D. Add a Parse JSON inside the flow.
E. Republish the agent.

Answer: A

Explanation:

A. Validate parameter definitions.

Aligning Mapped Variables: When a Copilot Studio agent calls a Power Automate flow, data exchange relies strictly on a contract of input and output variables. If the agent receives unexpected or empty values, it is typically because the parameter names, case sensitivity, or data types (e.g., text, number, boolean) defined in the flow's output action mismatch what the agent's calling node expects. Validating and matching these parameter definitions ensures data maps correctly at runtime.

Why the Other Options are incorrect:

B. Change the trigger schema: The trigger schema controls what data enters the flow from the agent, whereas the problem states that the agent is experiencing an issue receiving the returned structured values from the flow.
C. Review the flow run history: While useful for verifying if the flow completed successfully, it won't fix the schema binding contract mismatch that is causing the agent to drop or misunderstand the payload.
D. Add a Parse JSON inside the flow: Parse JSON is used inside a flow to handle unstructured data payloads received from an external API. It does not dictate how structured data properties are mapped back to the agent's expected output block.
E. Republish the agent: Republishing makes recent configuration changes live, but it will not resolve a structural data mismatch between the parameter bindings.

Question: 23 CertyIQ

A company is building an agent in Copilot Studio.
The agent must meet the following requirements:
Responses must be grounded in approved internal content.
The agent must be able to retrieve information from configured data sources in a topic.
You need to configure the agent so that topic responses are grounded from custom data sources.
What should you configure?

A. Enable analytics to review which topics users trigger most often.
B. Add a custom prompt that tells the agent to answer only from company data.
C. Configure knowledge sources in a generative answers node.
D. Add SharePoint as a knowledge source only at the agent level.
E. Increase the response length limit to improve answer completeness.

Answer: C

Explanation:

C. Configure knowledge sources in a generative answers node.

Topic-Level Grounding: To satisfy the requirement of retrieving information from custom data sources directly within a topic, you must use a **Generative answers node** inside that specific topic's authoring canvas.
Data Customization: Within the properties of that specific generative answers node, you can explicitly configure, scope, and override knowledge sources (such as specific SharePoint paths, public websites, or uploaded documents), ensuring the agent grounds its answers using that approved content during that specific conversational path.

Why the Other Options are Incorrect

A. Enable analytics: Analytics are used to monitor usage, track deflection rates, and find unhandled intents, but they have no functional impact on data grounding or information retrieval.
B. Add a custom prompt: While prompts guide an LLM's behavioral persona, they cannot securely retrieve or search through external, un-indexed custom data sources on demand without a retrieval mechanism.
D. Add SharePoint at the agent level only: The requirement explicitly states that the agent must retrieve information from data sources **in a topic**. Configuring it only at the agent level applies a global fallback search rather than target-scoped topic rules.
E. Increase the response length limit: This changes how much text the agent can output, but it does nothing to connect the agent to raw internal data repositories.

Question: 24 CertyIQ

DRAG DROP -
A company is building an agent in Copilot Studio topic that must retrieve real-time status information from an external REST API using a Send HTTP request node.
To meet the business needs, the topic must meet the following requirements:
The request must include the required authentication header.
The request must call the endpoint using the correct HTTP method.
The response must be configured with an appropriate response data type based on a schema and saved so the topic can reuse the returned values.
You need to configure the Send HTTP request node.
Which configuration should you use for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Add required headers to the HTTP request.
- Set the method to GET in the HTTP request.
- Select JSON data type and save response to a variable.
- Set the method to POST and send JSON in a request body.

HTTP Request node configuration

Requirement	Configuration
Configure authentication for the request.	Add required headers to the HTTP request.
Select the correct HTTP method for retrieving data.	Set the method to GET in the HTTP request.
Configure structured data handling and storage of data for reuse in the topic.	Select JSON data type and save response to a variable.

Answer:

Configurations

- Add required headers to the HTTP request.
- Set the method to GET in the HTTP request.
- Select JSON data type and save response to a variable.
- Set the method to POST and send JSON in a request body.

HTTP Request node configuration

Requirement	Configuration
Configure authentication for the request.	Add required headers to the HTTP request.
Select the correct HTTP method for retrieving data.	Set the method to GET in the HTTP request.
Configure structured data handling and storage of data for reuse in the topic.	Select JSON data type and save response to a variable.

Explanation:

1. Add required headers to the HTTP request.

Why it fits: When configuring custom raw HTTP actions in Copilot Studio, explicit authentication credentials (such as an API Key or Authorization: Bearer <token>) are manually included within the request's key-value metadata tables as specialized network **headers**.

2. Set the method to GET in the HTTP request.

Why it fits: Under standard REST API conventions, the **GET** HTTP verb is specifically designed and reserved for safely fetching or retrieving data from a target server without altering any state or records on the backend.

3. Select JSON data type and save response to a variable.

Why it fits: Handling data structures requires converting the raw text output from the API into structured objects. Selecting the **JSON** type tells the engine to parse the oncoming payload, while saving it into a **topic variable** preserves the details so subsequent nodes can read and reuse the data fields.

Question: 25

CertyIQ

A company is deploying an agent for employees.
The company has the following requirements:
Access must be restricted to the internal employees only.
End-user authentication must be enforced.
You need to configure the agent based on the company requirements.
What should you do?

A. Select Authenticate with Microsoft Entra ID for the agent.

B. Enable Require Secure Access in the Web channel security.

C. Publish the agent as a public website experience.

D. Select Authenticate with personal account for the agent.

Answer: A

Explanation:

A. Select Authenticate with Microsoft Entra ID for the agent.

Enterprise Security Standards: Selecting Authenticate with Microsoft Entra ID as the identity provider ensures that only individuals with a valid account inside your organization's tenant can log in and interact with the agent.
Access Control: Enabling this setting automatically prompts end-users to sign in using their corporate credentials before the conversation can begin. This setup works seamlessly with enterprise single sign-on (SSO), conditional access, and multi-factor authentication (MFA) policies.

Why the other options are incorrect:

Option B (Require Secure Access in the Web channel) handles token exchange security for specific web hosting environments but does not inherently restrict identity access control to specific internal tenant users.
Option C (Public website experience) explicitly violates the requirement by exposing the agent to the open internet.
Option D (Personal account) relies on consumer MSA accounts (like Outlook or Xbox logins) rather than the organization's enterprise tenant directory.

Question: 26 CertyIQ

DRAG DROP -
A company is integrating an agent in Copilot Studio into a custom application by using the Microsoft 365 Agents SDK.
The company has the following requirements for the application:
The deployment workflow must follow a dependency driven sequence.
The agent must be published before configuration values are retrieved.
End-to-end validation must occur before production deployment.
You need to plan the deployment workflow to integrate the agent.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

Deployment workflow steps	Answer Area	
Configure and enable the agent's target integration channel.	1	Configure and enable the agent's target integration channel.
Retrieve current agent configuration values.	2	Publish the agent.
Deploy the web app to production.	3	Retrieve current agent configuration values.
Implement the Microsoft 365 Agents SDK in the app using the agent configuration.	4	Implement the Microsoft 365 Agents SDK in the app using the agent configuration.
Publish the agent.	5	Validate the end-to-end conversation workflow in a test environment.
Validate the end-to-end conversation workflow in a test environment.	6	Deploy the web app to production.

Answer:

Deployment workflow steps	Answer Area	
Configure and enable the agent's target integration channel.	1	Publish the agent.
Retrieve current agent configuration values.	2	Configure and enable the agent's target integration channel.
Deploy the web app to production.	3	Retrieve current agent configuration values.
Implement the Microsoft 365 Agents SDK in the app using the agent configuration.	4	Implement the Microsoft 365 Agents SDK in the app using the agent configuration.
Publish the agent.	5	Validate the end-to-end conversation workflow in a test environment.
Validate the end-to-end conversation workflow in a test environment.	6	Deploy the web app to production.

Explanation:

1. **Publish the agent**: Any structural dialog models, topics, or configurations authored inside the studio workspace must be compiled and made live before channel integrations can successfully route requests.
2. **Configure and enable the channel**: Navigate to the agent's security and channel configuration parameters to activate the target portal (such as a custom web app or token routing endpoint).
3. **Retrieve configuration values**: Gather the distinct operational parameters exposed by your newly activated channel setup, such as the **Bot ID**, **Tenant ID**, and **Token Endpoint URL**.
4. **Implement the SDK**: Inject these target identifier values into your custom web application's codebase using the **Microsoft 365 Agents SDK** to initialize the real-time chat payload handshakes.
5. **Validate in a test environment**: Run comprehensive quality assurance cycles in a sandboxed staging or developer environment to guarantee that user tokens pass securely and the conversation executes correctly end-to-end.
6. **Deploy to production**: Once the integration code is thoroughly validated, push the finalized container or custom web application build into your live production environment for public consumption.

Question: 27 CertyIQ

An agent in Copilot Studio already exists in a development environment.
You need to prepare the agent so it can be moved to a production environment using the supported solution-based approach.
You need to create an exportable package that contains the existing agent.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Remove agent topics directly from the solution to prevent export errors.
B. Convert the solution to a managed solution so it can be exported.
C. Export the unmanaged solution that contains the agent.
D. Publish the agent to a channel and export the channel configuration.
E. Create a custom solution and add the existing agent to the solution.

Answer: CE

Explanation:

Add Existing Agent (E): Because the agent already exists standalone in your development environment, your first step is to create a new custom unmanaged solution container in the Maker portal. Then, you must use the **Add existing → Chatbot** (or Agent) option to bring the agent and all its subcomponents into this solution.
Exporting the Solution (C): Once the agent is bundled within your unmanaged custom solution container, you must export that unmanaged solution. The system will prompt you during this wizard to download it as a **managed** zip package, which is the official format for deployment into a downstream production environment.

Why the other options are incorrect:

Option A is incorrect because removing topics from the solution will break the agent, preventing it from functioning correctly in the production environment.
Option B is incorrect because solutions exist as unmanaged in the source environment; you do not "convert" it inside the development environment before exporting. The conversion to a managed file format happens during the export wizard process.
Option D is incorrect because exporting a channel configuration does not package the actual underlying logic, triggers, or framework of the agent itself.

Question: 28 CertyIQ

HOTSPOT -
A company wants to use content from a non-Microsoft enterprise knowledge base to ground agent responses.
To meet the business needs, the knowledge source must meet the following requirements:
Support grounding from indexed data.
Use existing access controls from the source system.
Support read-only operations.
You need to determine the correct behavior to meet the requirements.
Which behavior meets each requirement? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Copilot connector knowledge behavior

Requirement	Configuration
Support grounding from indexed data.	Retrieve indexed content Execute runtime operations. Retrieve live data at runtime.
Use existing access controls from the source system.	Respect existing permissions. Bypass permissions for faster retrieval. Respect existing permissions from agent authors.
Support read-only operations.	Prevent transactional updates. Allow authenticated write operations. Enable updates for enhanced performance.

Answer:

Copilot connector knowledge behavior

Requirement	Configuration
Support grounding from indexed data.	Retrieve indexed content Execute runtime operations. Retrieve live data at runtime.
Use existing access controls from the source system.	Respect existing permissions. Bypass permissions for faster retrieval. Respect existing permissions from agent authors.
Support read-only operations.	Prevent transactional updates. Allow authenticated write operations. Enable updates for enhanced performance.

Explanation:

1. Retrieve indexed content

Why it fits: When an agent grounds its generative responses using indexed data, it relies on a pre-built search catalog. Choosing **Retrieve indexed content** instructs the engine to look up data that has already been crawled and compiled for semantic discovery.

2. Respect existing permissions.

Why it fits: To ensure data compliance and security, the agent must never bypass security boundaries or display information a user isn't legally allowed to see. Selecting **Respect existing permissions.** ensures that access control lists (ACLs) and security trimming from the original host repository are strictly enforced.

3. Prevent transactional updates.

Why it fits: A read-only operational boundary strictly blocks mutations, data inputs, or overrides to the source material. Selecting **Prevent transactional updates.** explicitly drops or restricts writing permissions, keeping the connection entirely read-only.

Question: 29 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
An agent uses a flow that calls an external service which can occasionally fail or time out.
When a failure occurs, the agent must meet the following requirements:
Must not terminate silently.
Must send a notification containing the error details.
You need to configure the agent flow so that failures are handled in a controlled and predictable way.
Solution: Retry the failing action without handling failure.
Does the solution meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

A. Yes — INCORRECT

Why: A retry policy only repeats an action during transient network errors.

If all retry attempts are exhausted, the action simply reports Failed or TimedOut.

Because failure handling was intentionally skipped, execution stops dead in its tracks—violating both requirements.

B. No — CORRECT

Why: A retry policy is a resilience tactic, not an error-handling architecture.

Capturing runtime error data and dispatching alerts requires structured exception handling downstream.

Exam Concept: The AB-620 Standard Pattern

To satisfy both criteria in Power Automate / Copilot Studio flows, implement a Try-Catch Scope pattern:

Try Scope: Houses the external API/service call (with retry policy enabled for transient faults).

Catch Scope: Configured via "Configure run after" to execute only if the Try Scope has failed or has timed out.

Notification Action: Inside the Catch scope, extract the error using result('Try_Scope') and alert administrators via email or Microsoft Teams.

Question: 30 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
An agent uses a flow that calls an external service which can occasionally fail or time out.
When a failure occurs, the agent must meet the following requirements:
Must not terminate silently.
Must send a notification containing the error details.
You need to configure the agent flow so that failures are handled in a controlled and predictable way.
Solution: Configure run after conditions for failure paths.
Does the solution meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

Controlled Failure Paths: In Power Automate (used by Copilot Studio agent flows), configuring **"Run after"** conditions is the native and correct way to handle errors and timeouts. By default, actions only run if the previous step is successful. [1, 2, 3, 4, 5]
Capturing and Notifying Error Details: By adding a notification action (such as sending an email or posting a message) immediately following the external service node and changing its "Run after" behavior to fire if the previous action **"has failed"** or **"has timed out"**, you ensure the flow catches the exception. This directly fulfills both requirements: the flow handles the error predictably without terminating silently, and it allows you to dynamically extract and pass the specific error details into your notification layout. [1]

Question: 31 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
An agent uses a flow that calls an external service which can occasionally fail or time out.
When a failure occurs, the agent must meet the following requirements:
Must not terminate silently.
Must send a notification containing the error details.
You need to configure the agent flow so that failures are handled in a controlled and predictable way.
Solution: Send a notification with relevant error details.
Does the solution meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No.

Simply stating that you will "send a notification with relevant error details" is a functional description of the outcome, but it **does not configure the flow to handle the failure**.

Question: 32 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A company is configuring an agent in Copilot Studio that uses a generative answers node inside multiple topics.
The company requires responses to meet the following requirements:
Must use an executive summary format.
Must apply only within a specific topic.
Must continue using configured knowledge sources.
Must comply with connector security controls.
You need to configure a custom prompt to meet the formatting requirements.
Solution: Remove the generative answers node and rely only on the custom prompt to produce formatted responses.
Does the solution meet the goal?

A. Yes
B. No

Generative Answers vs. Custom Prompts (AB-620T00 — Integrate and extend agents in Copilot Studio / Use custom prompts with generative answers): In Microsoft Copilot Studio, the Create generative answers node is the core orchestration component responsible for querying configured enterprise knowledge sources (SharePoint, Dataverse, Copilot connectors, real-time connectors) while enforcing end-user authentication and connector security trimming.

Answer: B

Explanation:

B. No

Formatting Without Breaking Grounding: To enforce a specific output structure—such as an executive summary format—within a single topic without losing knowledge grounding, makers configure Custom instructions (prompt modification) directly inside that topic's Create generative answers node (or pass the grounded output of the generative answers node into a downstream Custom Prompt tool), rather than deleting the generative answers node altogether.

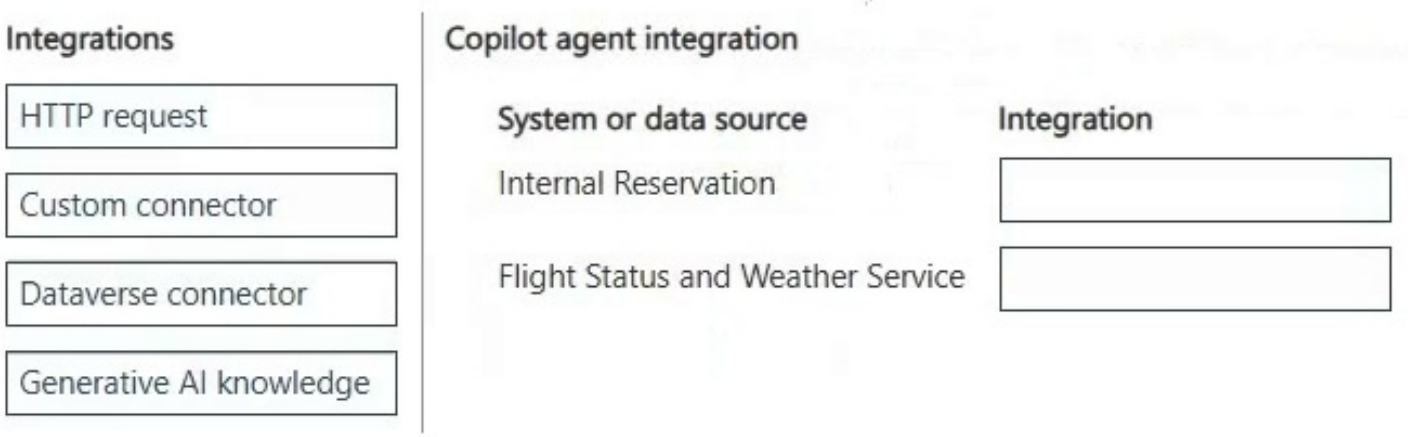
Losing Knowledge Connections: The requirements state that the agent must continue using configured knowledge sources. The **Generative answers node** is the core component in Copilot Studio responsible for handling information retrieval and querying those external knowledge bases.
Breaking Functionality: If you completely remove the generative answers node, the agent loses its bridge to search and fetch the required content. A standalone custom prompt tool can only format text using its base training data; it cannot independently query or reference the agent's configured live knowledge data sources.

Question: 33 CertyIQ

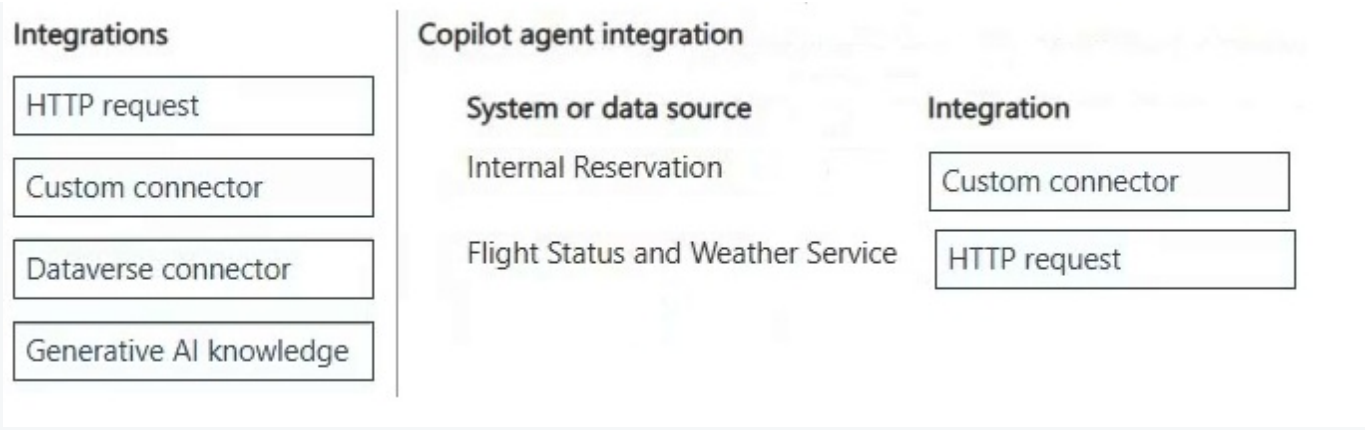
DRAG DROP -
Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -

The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to determine how the agent in Copilot Studio should integrate with two different systems in the Blue Yonder environment, based on their characteristics and the tools available.
Which integration mechanism should you use for each system? To answer, move the appropriate integrations to the correct system or data sources. You may use each integration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Internal Reservation → Custom connector

Why it fits: An internal reservation system is a proprietary business application unique to an organization. To wrap its custom API endpoints into a reusable, structured tool interface within Copilot Studio and the Power Platform ecosystem, developers build a **Custom connector**. This allows the agent to safely execute specific operations like creating or updating bookings.

2. Flight Status and Weather Service → HTTP request

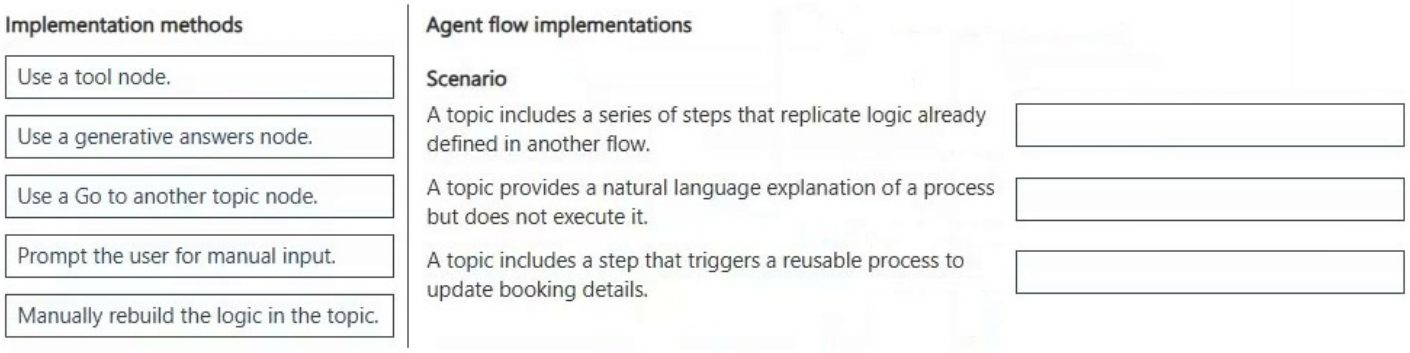
Why it fits: Public or standard web-hosted transactional utilities (such as dynamic flight status trackers or live weather web services) expose real-time endpoints. When you need to pull transient or highly dynamic status updates on demand during a conversation, sending a direct **HTTP request** is the most efficient approach.

Question: 34 CertyIQ

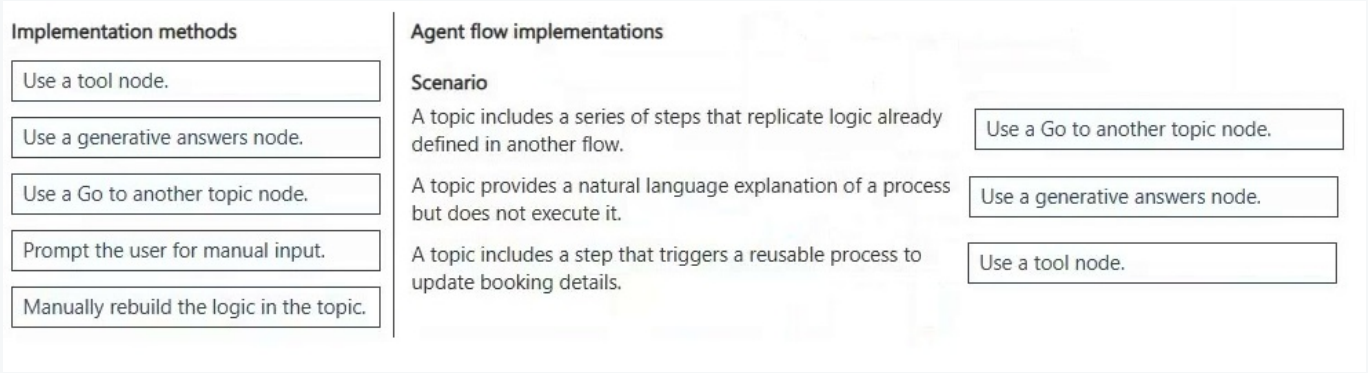
DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human

agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.
Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -

All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to determine an appropriate implementation related to the use of agent flows that is based on Blue Vender's design and compliance standards.
Which implementation method should you use for each scenario? To answer, move the appropriate implementation methods to the correct scenarios. You may use each implementation method once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Use a Go to another topic node.

Why it fits: To adhere to software design principles and maintain a clean conversational architecture, you should avoid replicating the same sequence of dialog actions across multiple places. If a conversation flow needs to reuse a sequence that has already been created elsewhere, you redirect the conversation path using a **Go to another topic node**.

2. Use a generative answers node.

Why it fits: When a user asks how a process works, and the agent only needs to explain or summarize information extracted from documentation or manuals rather than actually performing a database transaction, a **generative answers node** is used. It extracts and summarizes the text dynamically using foundational AI models.

3. Use a tool node.

Why it fits: Executing a core transactional modification in an external system (such as changing a status, modifying dates, or writing data to a booking repository) requires an active workflow execution. In Copilot Studio's unified canvas, calling these reusable operational processes is handled via a **tool node**.

Question: 35 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Fabrikam Inc. is a Canada-based manufacturer with a growing service organization that supports field technicians and internal operations teams. Fabrikam Inc. plans to launch a new internal agent solution named Operations Concierge to reduce time spent searching policy content, retrieving operational metrics, and executing routine transactions.
The agent will be used by three groups:
Service coordinators who triage incoming service requests
Field technicians who need guided procedures and parts availability
Operations managers who monitor KPIs and exceptions
The agent solution must work in real-world operational conditions. Users often ask questions mid-call with a customer or while coordinating parts shipments. The agents require quick, reliable outcomes. As a result, Fabrikam Inc. requires the solution to:
Provide grounded answers with traceability when it provides guidance.
Retrieve real-time metrics when users ask for operational status.
Execute authenticated updates when users initiate a flow (such as creating a parts request).
Fabrikam Inc. also expects the solution to be maintained by multiple makers and developers across the year. The company has experienced duplicated logic and inconsistent behavior across different agents. This project emphasizes reuse, governance, and maintainability across teams.
Current environment -
Fabrikam Inc. runs three Microsoft Power Platform environments for agent development and release: Dev, Test, and Prod.
The team plans to build the agent and validate it in Dev and Test, then promote to Prod by using a controlled release process that supports repeatable deployments.
Fabrikam Inc. already has two assets the team wants to reuse:
A partially completed Copilot Studio agent named Service Desk Agent, used by IT to create internal tickets and route requests
A Microsoft Foundry agent created by a central AI team that performs specialized summarization and classification for long-form text (for example, summarizing call transcripts into an incident narrative)
Fabrikam Inc. also has operational and knowledge data sources:

A curated policy library (internal SOPs, service warranty rules, escalation criteria, and standard operating procedures)
A set of indexed documents and procedures in an Azure AI Search service that supports vector search for the policy library
A Microsoft Fabric workspace that includes a semantic model used by operations leadership for reporting
Business requirements -
Fabrikam Inc. requires Operations Concierge to meet the following business requirements:
Traceability requirement: When the agent provides policy guidance or procedural recommendations, users must be able to see where the answer came from.
Metrics requirement: When users ask about service performance (backlog, SLA risk, parts shortages, dispatch delays), the solution must return up-to-date metrics in a structured format that operations managers can use in weekly reviews.
Transaction requirement: The solution must support authenticated updates initiated during conversations, including creating a parts request and updating a service case status.
In addition, Fabrikam Inc. wants to avoid duplicating common assets across agents:
The team must reuse the same set of escalation topics, MCP tool definitions, and a standard safety disclaimer across three different agents.
Only the platform engineering group as allowed to edit shared assets. However, all agent authors must be able to use them.
Technical requirements -
The Fabrikam Inc. solution architecture uses a multi-agent approach so that specialist responsibilities are isolated and can evolve independently.
The Operations Concierge (primary agent) must coordinate the following specialist capabilities:
Policy and procedure Q&A: Use an enterprise knowledge source that supports indexed retrieval across the curated policy library and service procedures.
Operational metrics: Delegate metric queries to a Fabric Data Agent that reads governed business data through the Fabric semantic model.
Authenticated updates: Use tools exposed by an existing internal Model Context Protocol (MCP) server that provides transactional operations for the service organization.
Specialized processing: Delegate summarization and classification requests to an existing Microsoft Foundry agent.
Fabrikam Inc. will onboard two MCP servers as tools:
PartsOps MCP server: exposes tools for parts availability checks and parts request creation. The server requires per-user authentication because actions must be traceable to the requesting user.
WarrantyRules MCP server: exposes a read-only tool for validating warranty coverage. The server uses an API key shared by the agent team.
Fabrikam Inc. has also defined a collaboration requirement with the existing Service Desk Agent:
The primary agent must delegate IT-specific requests to the existing Service Desk Agent rather than reimplement ticket creation logic.
Finally, Fabrikarn Inc. plans to support a partner integration:
For shipment tracking inquiries, Fabrikam Inc. will delegate to a partner-provided agent that is only available through a standardized agent-to-agent endpoint.
Issues and constraints -
During early testing, Fabrikam Inc. found three recurring problems:
Makers are copying and modifying the same components across agents, resulting in inconsistent disclaimers and duplicated tools.
Users can obtain a correct answer, but the response is not consistently traceable to a source when the agent uses knowledge.
The primary agent can route some requests, but specialist capabilities are not consistently delegated (for example, some metric questions are answered generatively instead of being routed to the Fabric Data Agent).
You are part of the engineering team responsible for correcting the design and configuration to meet the preceding requirements and constraints.
You need to enable Operations Concierge to delegate shipment tracking inquiries according to Fabrikam Inc.'s defined architecture and technical requirements.
Which integration approach should you use?

A. Configure the partner capability as an enterprise knowledge source with indexed retrieval enabled.

B. Configure the partner capability as an MCP tool and invoke it as a transactional operation.

C. Configure the partner capability as an A2A integration and supply its endpoint information.

D. Configure the primary agent to handle shipment tracking through generative responses after publishing.

Answer: C

Explanation:

C. Configure the partner capability as an A2A integration and supply its endpoint information.

Resolving Delegation Failures: The third constraint explicitly notes that the primary agent is failing to delegate requests to specialist capabilities correctly (instead relying on general generative answers). Implementing an **Agent-to-Agent (A2A)** integration and supplying the destination endpoint is the native Copilot Studio mechanism designed to establish multi-agent routing boundaries [C].
Seamless Hand-offs: This configuration registers the partner's specialized system as a distinct external agent. The primary orchestrator (Operations Concierge) can then seamlessly delegate conversational control and route shipment tracking inquiries to that specialist agent rather than trying to summarize it broadly using base model knowledge.

Question: 36 CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Fabrikam Inc. is a Canada-based manufacturer with a growing service organization that supports field technicians and internal operations teams. Fabrikam Inc. plans to launch a new internal agent solution named Operations Concierge to reduce time spent searching policy content, retrieving operational metrics, and executing routine transactions.
The agent will be used by three groups:
Service coordinators who triage incoming service requests
Field technicians who need guided procedures and parts availability
Operations managers who monitor KPIs and exceptions
The agent solution must work in real-world operational conditions. Users often ask questions mid-call with a customer or while coordinating parts shipments. The agents require quick, reliable outcomes. As a result, Fabrikam Inc. requires the solution to:
Provide grounded answers with traceability when it provides guidance.
Retrieve real-time metrics when users ask for operational status.
Execute authenticated updates when users initiate a flow (such as creating a parts request).
Fabrikam Inc. also expects the solution to be maintained by multiple makers and developers across the year. The company has experienced duplicated logic and inconsistent behavior across different agents. This project emphasizes reuse, governance, and maintainability across teams.
Current environment -
Fabrikam Inc. runs three Microsoft Power Platform environments for agent development and release: Dev, Test, and Prod.
The team plans to build the agent and validate it in Dev and Test, then promote to Prod by using a controlled release process that supports repeatable deployments.
Fabrikam Inc. already has two assets the team wants to reuse:
A partially completed Copilot Studio agent named Service Desk Agent, used by IT to create internal tickets and

route requests
A Microsoft Foundry agent created by a central AI team that performs specialized summarization and classification for long-form text (for example, summarizing call transcripts into an incident narrative)
Fabrikam Inc. also has operational and knowledge data sources:
A curated policy library (internal SOPs, service warranty rules, escalation criteria, and standard operating procedures)
A set of indexed documents and procedures in an Azure AI Search service that supports vector search for the policy library
A Microsoft Fabric workspace that includes a semantic model used by operations leadership for reporting
Business requirements -
Fabrikam Inc. requires Operations Concierge to meet the following business requirements:
Traceability requirement: When the agent provides policy guidance or procedural recommendations, users must be able to see where the answer came from.
Metrics requirement: When users ask about service performance (backlog, SLA risk, parts shortages, dispatch delays), the solution must return up-to-date metrics in a structured format that operations managers can use in weekly reviews.
Transaction requirement: The solution must support authenticated updates initiated during conversations, including creating a parts request and updating a service case status.
In addition, Fabrikam Inc. wants to avoid duplicating common assets across agents:
The team must reuse the same set of escalation topics, MCP tool definitions, and a standard safety disclaimer across three different agents.
Only the platform engineering group as allowed to edit shared assets. However, all agent authors must be able to use them.
Technical requirements -
The Fabrikam Inc. solution architecture uses a multi-agent approach so that specialist responsibilities are isolated and can evolve independently.
The Operations Concierge (primary agent) must coordinate the following specialist capabilities:
Policy and procedure Q&A: Use an enterprise knowledge source that supports indexed retrieval across the curated policy library and service procedures.
Operational metrics: Delegate metric queries to a Fabric Data Agent that reads governed business data through the Fabric semantic model.
Authenticated updates: Use tools exposed by an existing internal Model Context Protocol (MCP) server that provides transactional operations for the service organization.
Specialized processing: Delegate summarization and classification requests to an existing Microsoft Foundry agent.
Fabrikam Inc. will onboard two MCP servers as tools:
PartsOps MCP server: exposes tools for parts availability checks and parts request creation. The server requires per-user authentication because actions must be traceable to the requesting user.
WarrantyRules MCP server: exposes a read-only tool for validating warranty coverage. The server uses an API key shared by the agent team.
Fabrikam Inc. has also defined a collaboration requirement with the existing Service Desk Agent:
The primary agent must delegate IT-specific requests to the existing Service Desk Agent rather than reimplement ticket creation logic.
Finally, Fabrikarn Inc. plans to support a partner integration:
For shipment tracking inquiries, Fabrikam Inc. will delegate to a partner-provided agent that is only available through a standardized agent-to-agent endpoint.
Issues and constraints -
During early testing, Fabrikam Inc. found three recurring problems:
Makers are copying and modifying the same components across agents, resulting in inconsistent disclaimers and duplicated tools.
Users can obtain a correct answer, but the response is not consistently traceable to a source when the agent uses knowledge.
The primary agent can route some requests, but specialist capabilities are not consistently delegated (for example, some metric questions are answered generatively instead of being routed to the Fabric Data Agent).
You are part of the engineering team responsible for correcting the design and configuration to meet the preceding requirements and constraints.
You need to integrate Fabrikam Inc.'s existing Foundry agent so Operations Concierge can delegate summarization requests.
Which action should you perform for each requirement? To answer, move the appropriate actions to the correct requirements. You may use each action once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Actions

- Add an external agent of type Microsoft Foundry.
- Publish the Copilot Studio agent to enable external calls.
- Add the Foundry output as an indexed knowledge source.
- Enter the Foundry project endpoint URL to create the connection.

Foundry agent integrations

Requirement	Action
The main agent must be able to call an external Foundry agent.	
The external Foundry agent must reference the correct Foundry project and agent.	

Answer:

Actions

- Add an external agent of type Microsoft Foundry.
- Publish the Copilot Studio agent to enable external calls.
- Add the Foundry output as an indexed knowledge source.
- Enter the Foundry project endpoint URL to create the connection.

Foundry agent integrations

Requirement	Action
The main agent must be able to call an external Foundry agent.	Add an external agent of type Microsoft Foundry.
The external Foundry agent must reference the correct Foundry project and agent.	Enter the Foundry project endpoint URL to create the connection.

Explanation:

1. Add an external agent of type Microsoft Foundry.

Why it fits: When establishing a collaborative multi-agent architecture where a primary orchestrator copilot delegates specific sub-tasks to a specialized external agent created in Azure AI Foundry, you navigate to the **Agents** tab in Microsoft Copilot Studio and select **Microsoft Foundry** from the available external agent connection list.

2. Enter the Foundry project endpoint URL to create the connection.

Why it fits: To ensure the external agent connection points directly to your targeted environment and specific model resources, the configuration requires establishing a handshake via the unique network backbone. Supplying the precise project endpoint or connection string bridges the platform boundary to register the designated project container.

Question: 37

CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to

return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Fabrikam Inc. is a Canada-based manufacturer with a growing service organization that supports field technicians and internal operations teams. Fabrikam Inc. plans to launch a new internal agent solution named Operations Concierge to reduce time spent searching policy content, retrieving operational metrics, and executing routine transactions.
The agent will be used by three groups:
Service coordinators who triage incoming service requests
Field technicians who need guided procedures and parts availability
Operations managers who monitor KPIs and exceptions
The agent solution must work in real-world operational conditions. Users often ask questions mid-call with a customer or while coordinating parts shipments. The agents require quick, reliable outcomes. As a result, Fabrikam Inc. requires the solution to:
Provide grounded answers with traceability when it provides guidance.
Retrieve real-time metrics when users ask for operational status.
Execute authenticated updates when users initiate a flow (such as creating a parts request).
Fabrikam Inc. also expects the solution to be maintained by multiple makers and developers across the year. The company has experienced duplicated logic and inconsistent behavior across different agents. This project emphasizes reuse, governance, and maintainability across teams.
Current environment -
Fabrikam Inc. runs three Microsoft Power Platform environments for agent development and release: Dev, Test, and Prod.
The team plans to build the agent and validate it in Dev and Test, then promote to Prod by using a controlled release process that supports repeatable deployments.
Fabrikam Inc. already has two assets the team wants to reuse:
A partially completed Copilot Studio agent named Service Desk Agent, used by IT to create internal tickets and route requests
A Microsoft Foundry agent created by a central AI team that performs specialized summarization and classification for long-form text (for example, summarizing call transcripts into an incident narrative)
Fabrikam Inc. also has operational and knowledge data sources:
A curated policy library (internal SOPs, service warranty rules, escalation criteria, and standard operating procedures)
A set of indexed documents and procedures in an Azure AI Search service that supports vector search for the policy library
A Microsoft Fabric workspace that includes a semantic model used by operations leadership for reporting
Business requirements -
Fabrikam Inc. requires Operations Concierge to meet the following business requirements:
Traceability requirement: When the agent provides policy guidance or procedural recommendations, users must be able to see where the answer came from.
Metrics requirement: When users ask about service performance (backlog, SLA risk, parts shortages, dispatch delays), the solution must return up-to-date metrics in a structured format that operations managers can use in weekly reviews.
Transaction requirement: The solution must support authenticated updates initiated during conversations, including creating a parts request and updating a service case status.
In addition, Fabrikam Inc. wants to avoid duplicating common assets across agents:
The team must reuse the same set of escalation topics, MCP tool definitions, and a standard safety disclaimer across three different agents.
Only the platform engineering group as allowed to edit shared assets. However, all agent authors must be able to use them.
Technical requirements -
The Fabrikam Inc. solution architecture uses a multi-agent approach so that specialist responsibilities are isolated and can evolve independently.
The Operations Concierge (primary agent) must coordinate the following specialist capabilities:
Policy and procedure Q&A: Use an enterprise knowledge source that supports indexed retrieval across the curated policy library and service procedures.
Operational metrics: Delegate metric queries to a Fabric Data Agent that reads governed business data through the Fabric semantic model.
Authenticated updates: Use tools exposed by an existing internal Model Context Protocol (MCP) server that provides transactional operations for the service organization.
Specialized processing: Delegate summarization and classification requests to an existing Microsoft Foundry agent.

Fabrikam Inc. will onboard two MCP servers as tools:
PartsOps MCP server: exposes tools for parts availability checks and parts request creation. The server requires per-user authentication because actions must be traceable to the requesting user.
WarrantyRules MCP server: exposes a read-only tool for validating warranty coverage. The server uses an API key shared by the agent team.
Fabrikam Inc. has also defined a collaboration requirement with the existing Service Desk Agent:
The primary agent must delegate IT-specific requests to the existing Service Desk Agent rather than reimplement ticket creation logic.
Finally, Fabrikarn Inc. plans to support a partner integration:
For shipment tracking inquiries, Fabrikam Inc. will delegate to a partner-provided agent that is only available through a standardized agent-to-agent endpoint.
Issues and constraints -
During early testing, Fabrikam Inc. found three recurring problems:
Makers are copying and modifying the same components across agents, resulting in inconsistent disclaimers and duplicated tools.
Users can obtain a correct answer, but the response is not consistently traceable to a source when the agent uses knowledge.
The primary agent can route some requests, but specialist capabilities are not consistently delegated (for example, some metric questions are answered generatively instead of being routed to the Fabric Data Agent).
You are part of the engineering team responsible for correcting the design and configuration to meet the preceding requirements and constraints.
You need to plan how Fabrikam Inc. will reuse shared components across multiple agents while limiting who can change the shared components.
Which approach should you use for each requirement? To answer, move the appropriate components to the correct requirements. You may use each component once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Components

- Restrict component collection to the primary agent.
- Create a component collection and move shared components into it.
- Export the primary agent as a solution and copy assets into other agents.
- Share the component collection with teammates while controlling edit access.

Reusable components

Requirement	Component
Prevent shared assets from being modified by most builders.	
Reuse the same set of components across multiple agents without copying.	
Limit a shared set of components so it can be used only by its intended primary agent.	

Answer:

Components

- Restrict component collection to the primary agent.
- Create a component collection and move shared components into it.
- Export the primary agent as a solution and copy assets into other agents.
- Share the component collection with teammates while controlling edit access.

Reusable components

Requirement	Component
Prevent shared assets from being modified by most builders.	Share the component collection with teammates while controlling edit access.
Reuse the same set of components across multiple agents without copying.	Create a component collection and move shared components into it.
Limit a shared set of components so it can be used only by its intended primary agent.	Restrict component collection to the primary agent.

Explanation:

1. Share the component collection with teammates while controlling edit access.

Why it fits: When a team shares building blocks (like prompts, knowledge schemas, or dialog models) but wants to protect them from unintended alterations by secondary makers, they utilize fine-grained sharing controls. Granting read-only or run-only permissions to the collection prevents unauthorized modifications.

2. Create a component collection and move shared components into it.

Why it fits: Instead of manually duplicating individual topics or logic chains across several independent chatbots — which leads to an unmaintainable codebase — builders centralize these common assets into a singular **component collection**. This collection serves as a shared library that multiple agents link to directly without duplication.

3. Restrict component collection to the primary agent.

Why it fits: If a highly specialized group of reusable assets is strictly intended for a single core agent, you configure its distribution settings to limit consumption boundaries. Restricting the collection to only the designated primary orchestrator prevents other agents in the environment from picking it up.

Question: 38 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Fabrikam Inc. is a Canada-based manufacturer with a growing service organization that supports field technicians and internal operations teams. Fabrikam Inc. plans to launch a new internal agent solution named Operations Concierge to reduce time spent searching policy content, retrieving operational metrics, and executing routine transactions.
The agent will be used by three groups:
Service coordinators who triage incoming service requests
Field technicians who need guided procedures and parts availability
Operations managers who monitor KPIs and exceptions
The agent solution must work in real-world operational conditions. Users often ask questions mid-call with a customer or while coordinating parts shipments. The agents require quick, reliable outcomes. As a result, Fabrikam Inc. requires the solution to:
Provide grounded answers with traceability when it provides guidance.
Retrieve real-time metrics when users ask for operational status.
Execute authenticated updates when users initiate a flow (such as creating a parts request).
Fabrikam Inc. also expects the solution to be maintained by multiple makers and developers across the year. The company has experienced duplicated logic and inconsistent behavior across different agents. This project emphasizes reuse, governance, and maintainability across teams.
Current environment -
Fabrikam Inc. runs three Microsoft Power Platform environments for agent development and release: Dev, Test, and Prod.
The team plans to build the agent and validate it in Dev and Test, then promote to Prod by using a controlled release process that supports repeatable deployments.
Fabrikam Inc. already has two assets the team wants to reuse:
A partially completed Copilot Studio agent named Service Desk Agent, used by IT to create internal tickets and route requests

A Microsoft Foundry agent created by a central AI team that performs specialized summarization and classification for long-form text (for example, summarizing call transcripts into an incident narrative)
Fabrikam Inc. also has operational and knowledge data sources:
A curated policy library (internal SOPs, service warranty rules, escalation criteria, and standard operating procedures)
A set of indexed documents and procedures in an Azure AI Search service that supports vector search for the policy library
A Microsoft Fabric workspace that includes a semantic model used by operations leadership for reporting
Business requirements -
Fabrikam Inc. requires Operations Concierge to meet the following business requirements:
Traceability requirement: When the agent provides policy guidance or procedural recommendations, users must be able to see where the answer came from.
Metrics requirement: When users ask about service performance (backlog, SLA risk, parts shortages, dispatch delays), the solution must return up-to-date metrics in a structured format that operations managers can use in weekly reviews.
Transaction requirement: The solution must support authenticated updates initiated during conversations, including creating a parts request and updating a service case status.
In addition, Fabrikam Inc. wants to avoid duplicating common assets across agents:
The team must reuse the same set of escalation topics, MCP tool definitions, and a standard safety disclaimer across three different agents.
Only the platform engineering group as allowed to edit shared assets. However, all agent authors must be able to use them.
Technical requirements -
The Fabrikam Inc. solution architecture uses a multi-agent approach so that specialist responsibilities are isolated and can evolve independently.
The Operations Concierge (primary agent) must coordinate the following specialist capabilities:
Policy and procedure Q&A: Use an enterprise knowledge source that supports indexed retrieval across the curated policy library and service procedures.
Operational metrics: Delegate metric queries to a Fabric Data Agent that reads governed business data through the Fabric semantic model.
Authenticated updates: Use tools exposed by an existing internal Model Context Protocol (MCP) server that provides transactional operations for the service organization.
Specialized processing: Delegate summarization and classification requests to an existing Microsoft Foundry agent.
Fabrikam Inc. will onboard two MCP servers as tools:
PartsOps MCP server: exposes tools for parts availability checks and parts request creation. The server requires per-user authentication because actions must be traceable to the requesting user.
WarrantyRules MCP server: exposes a read-only tool for validating warranty coverage. The server uses an API key shared by the agent team.
Fabrikam Inc. has also defined a collaboration requirement with the existing Service Desk Agent:
The primary agent must delegate IT-specific requests to the existing Service Desk Agent rather than reimplement ticket creation logic.
Finally, Fabrikarn Inc. plans to support a partner integration:
For shipment tracking inquiries, Fabrikam Inc. will delegate to a partner-provided agent that is only available through a standardized agent-to-agent endpoint.
Issues and constraints -
During early testing, Fabrikam Inc. found three recurring problems:
Makers are copying and modifying the same components across agents, resulting in inconsistent disclaimers and duplicated tools.
Users can obtain a correct answer, but the response is not consistently traceable to a source when the agent uses knowledge.
The primary agent can route some requests, but specialist capabilities are not consistently delegated (for example, some metric questions are answered generatively instead of being routed to the Fabric Data Agent).
You are part of the engineering team responsible for correcting the design and configuration to meet the preceding requirements and constraints.
You need to configure Operations Concierge so that prompt-driven summarization behavior aligns with Fabrikam Inc.'s architectural requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Configure Azure AI Search as the grounding source for summarization prompts.

B. Publish the agent to ensure the selected model is enforced for summarization.

C. Select a model from the Foundry model catalog in the custom prompt configuration.

D. Enable citations to control how summarization output is generated.

E. Create a reusable custom prompt template and apply it where summarization responses are generated.

Answer: CE

Explanation:

C. Select a model from the Foundry model catalog in the custom prompt configuration.

E. Create a reusable custom prompt template and apply it where summarization responses are generated.

Why these are the correct choices:

Why C fits: To adhere to enterprise governance and ensure compliance with Fabrikam Inc.'s security standards, you must bypass the standard out-of-the-box model. Selecting a vetted, corporate-approved foundational model directly from the Azure AI Foundry catalog within the prompt configuration ensures that the summarization workload is processed strictly by authorized infrastructure.
Why E fits: Creating a structured, reusable custom prompt template ensures that formatting rules, tone parameters, and operational constraints are uniformly applied across all areas where summaries are generated. This template acts as the central definition of the agent's summarization behavior across the application lifecycle.

Question: 39 CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Fabrikam Inc. is a Canada-based manufacturer with a growing service organization that supports field technicians and internal operations teams. Fabrikam Inc. plans to launch a new internal agent solution named Operations Concierge to reduce time spent searching policy content, retrieving operational metrics, and executing routine transactions.
The agent will be used by three groups:
Service coordinators who triage incoming service requests
Field technicians who need guided procedures and parts availability
Operations managers who monitor KPIs and exceptions
The agent solution must work in real-world operational conditions. Users often ask questions mid-call with a customer or while coordinating parts shipments. The agents require quick, reliable outcomes. As a result, Fabrikam Inc. requires the solution to:
Provide grounded answers with traceability when it provides guidance.
Retrieve real-time metrics when users ask for operational status.
Execute authenticated updates when users initiate a flow (such as creating a parts request).

Fabrikam Inc. also expects the solution to be maintained by multiple makers and developers across the year. The company has experienced duplicated logic and inconsistent behavior across different agents. This project emphasizes reuse, governance, and maintainability across teams.

Current environment -

Fabrikam Inc. runs three Microsoft Power Platform environments for agent development and release: Dev, Test, and Prod.

The team plans to build the agent and validate it in Dev and Test, then promote to Prod by using a controlled release process that supports repeatable deployments.

Fabrikam Inc. already has two assets the team wants to reuse:

A partially completed Copilot Studio agent named Service Desk Agent, used by IT to create internal tickets and route requests

A Microsoft Foundry agent created by a central AI team that performs specialized summarization and classification for long-form text (for example, summarizing call transcripts into an incident narrative)

Fabrikam Inc. also has operational and knowledge data sources:

A curated policy library (internal SOPs, service warranty rules, escalation criteria, and standard operating procedures)

A set of indexed documents and procedures in an Azure AI Search service that supports vector search for the policy library

A Microsoft Fabric workspace that includes a semantic model used by operations leadership for reporting

Business requirements -

Fabrikam Inc. requires Operations Concierge to meet the following business requirements:

Traceability requirement: When the agent provides policy guidance or procedural recommendations, users must be able to see where the answer came from.

Metrics requirement: When users ask about service performance (backlog, SLA risk, parts shortages, dispatch delays), the solution must return up-to-date metrics in a structured format that operations managers can use in weekly reviews.

Transaction requirement: The solution must support authenticated updates initiated during conversations, including creating a parts request and updating a service case status.

In addition, Fabrikam Inc. wants to avoid duplicating common assets across agents:

The team must reuse the same set of escalation topics, MCP tool definitions, and a standard safety disclaimer across three different agents.

Only the platform engineering group as allowed to edit shared assets. However, all agent authors must be able to use them.

Technical requirements -

The Fabrikam Inc. solution architecture uses a multi-agent approach so that specialist responsibilities are isolated and can evolve independently.

The Operations Concierge (primary agent) must coordinate the following specialist capabilities:

Policy and procedure Q&A: Use an enterprise knowledge source that supports indexed retrieval across the curated policy library and service procedures.

Operational metrics: Delegate metric queries to a Fabric Data Agent that reads governed business data through the Fabric semantic model.

Authenticated updates: Use tools exposed by an existing internal Model Context Protocol (MCP) server that provides transactional operations for the service organization.

Specialized processing: Delegate summarization and classification requests to an existing Microsoft Foundry agent.

Fabrikam Inc. will onboard two MCP servers as tools:

PartsOps MCP server: exposes tools for parts availability checks and parts request creation. The server requires per-user authentication because actions must be traceable to the requesting user.

WarrantyRules MCP server: exposes a read-only tool for validating warranty coverage. The server uses an API key shared by the agent team.

Fabrikam Inc. has also defined a collaboration requirement with the existing Service Desk Agent:

The primary agent must delegate IT-specific requests to the existing Service Desk Agent rather than reimplement ticket creation logic.

Finally, Fabrikarn Inc. plans to support a partner integration:

For shipment tracking inquiries, Fabrikam Inc. will delegate to a partner-provided agent that is only available through a standardized agent-to-agent endpoint.

Issues and constraints -

During early testing, Fabrikam Inc. found three recurring problems:

Makers are copying and modifying the same components across agents, resulting in inconsistent disclaimers and duplicated tools.

Users can obtain a correct answer, but the response is not consistently traceable to a source when the agent uses knowledge.

The primary agent can route some requests, but specialist capabilities are not consistently delegated (for example, some metric questions are answered generatively instead of being routed to the Fabric Data Agent).

You are part of the engineering team responsible for correcting the design and configuration to meet the preceding requirements and constraints.

You need to connect Operations Concierge to Fabrikam Inc.'s Azure AI Search knowledge index while complying with security requirements.
Which configuration should you use for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Enable citations for grounded answers.
- Enter the name of the Azure AI Search index.
- Select the service principal as the authentication type.
- Provide the Azure AI Search Endpoint URL in the connection details.

Azure AI Search knowledge source connection

Requirement	Configuration
Ensure the Azure AI Search connection complies with the governance requirement for authenticated access.	
Ensure policy and procedure Q&A uses the required curated, indexed enterprise knowledge source.	
Ensure the connection targets the Azure AI Search service instance that stores the indexed policy documents.	

Answer:

Configurations

- Enable citations for grounded answers.
- Enter the name of the Azure AI Search index.
- Select the service principal as the authentication type.
- Provide the Azure AI Search Endpoint URL in the connection details.

Azure AI Search knowledge source connection

Requirement	Configuration
Ensure the Azure AI Search connection complies with the governance requirement for authenticated access.	Select the service principal as the authentication type.
Ensure policy and procedure Q&A uses the required curated, indexed enterprise knowledge source.	Enter the name of the Azure AI Search index.
Ensure the connection targets the Azure AI Search service instance that stores the indexed policy documents.	Provide the Azure AI Search Endpoint URL in the connection details.

Explanation:

1. Select the service principal as the authentication type.

Why it fits: Governance requirements for enterprise integrations typically rule out using shared keys or individual user accounts. Selecting a **service principal** provides a non-interactive, highly secure, and auditable identity layer to handle authentication between the agent and Azure AI Search.

2. Enter the name of the Azure AI Search index.

Why it fits: An individual Azure AI Search service instance can host many different document sets. Specifying the exact **index name** ensures that the agent targets the precise data store containing the curated policy and procedure files rather than looking through unrelated enterprise indexes.

3. Provide the Azure AI Search Endpoint URL in the connection details.

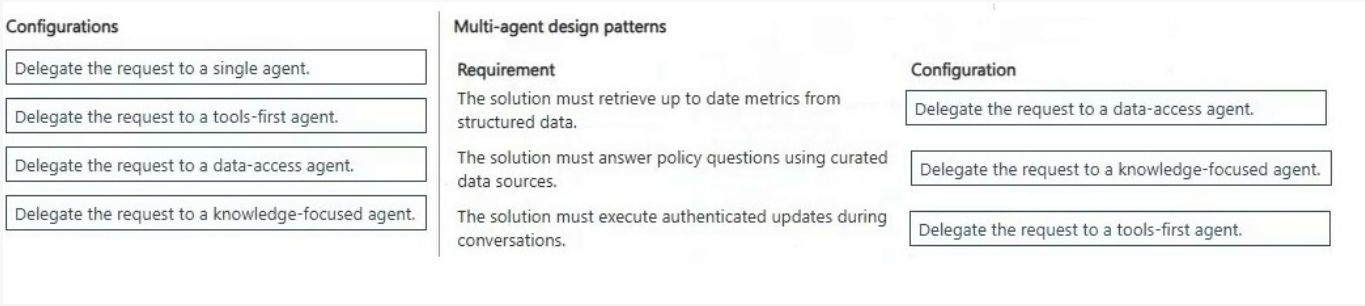
Why it fits: To navigate traffic to the correct cloud infrastructure instance where your search engine runs, you must provide its physical resource route. The **Endpoint URL** establishes the baseline connection to that specific service instance hosting your indexes.

Question: 40 CertyIQ

DRAG DROP -
A company is building a Copilot Studio solution that must answer policy questions, retrieve operational metrics from structured data, and execute authenticated updates in backend systems. The company decides to use

multiple specialized agents rather than a single monolithic agent. Custom agents for each action already exist.
To meet the business needs, the solution must meet the following requirements:
The solution must retrieve up to date metrics from structured data.
The solution must answer policy questions using curated knowledge.
The solution must execute authenticated updates during conversations.
You need to select the appropriate agent design for each requirement.
Which approach should you use for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Answer:



Explanation:

1. Delegate the request to a data-access agent.

Why it fits: When an agent needs to retrieve up-to-date business metrics, KPIs, or specific records stored inside organized relational databases or structured tables (like Microsoft Dataverse, SQL, or tabular systems), it delegates the task to a specialized **data-access agent** optimized for handling database queries and structured schemas.

2. Delegate the request to a knowledge-focused agent.

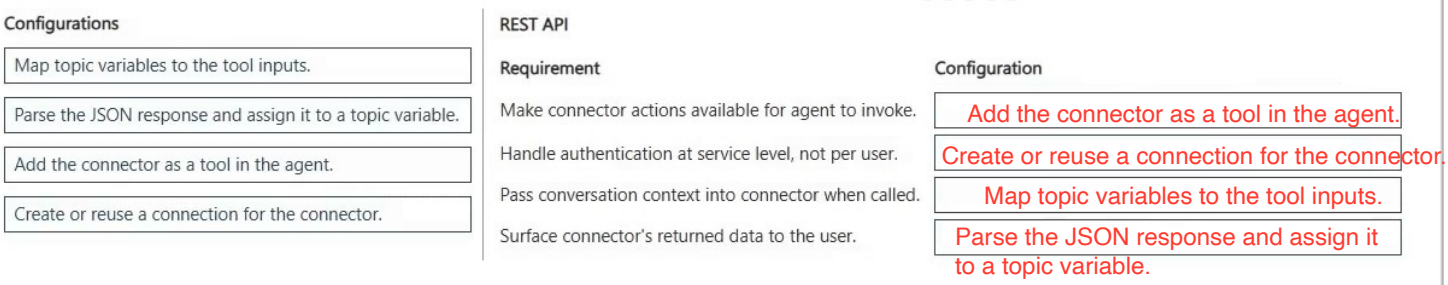
Why it fits: Answering policy, compliance, or procedural questions involves searching through unstructured text corpora (such as company handbooks, PDFs, and manuals). A **knowledge-focused agent** is specifically designed for Retrieval-Augmented Generation (RAG) and semantic exploration over curated text indexes.

3. Delegate the request to a tools-first agent.

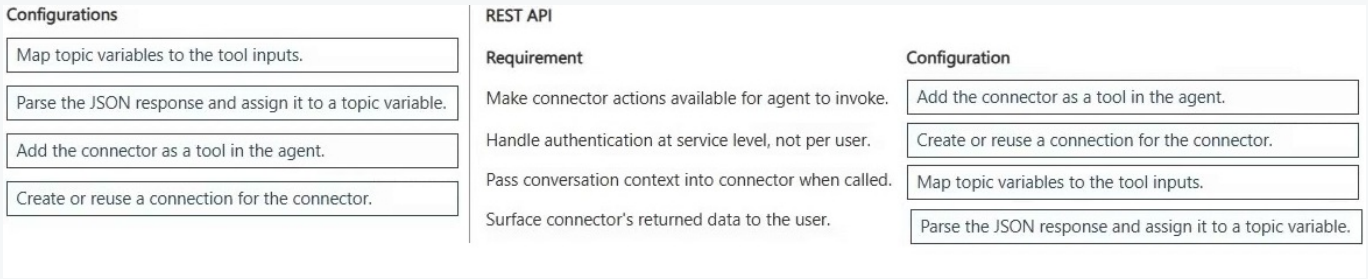
Why it fits: Executing data mutations or active operations (such as making modifications, triggering write workflows, or executing transactional updates) requires an agent architecture focused on API invocation. A **tools-first agent** specializes in discovering, configuring inputs for, and reliably triggering actions or connectors to write changes back to external source systems under an authenticated user context.

Question: 41 CertyIQ

DRAG DROP -
A company has an approved custom connector that uses a REST API for an internal system.
An agent in Copilot Studio must call this connector during conversations to retrieve or update data.
To meet the business needs, the solution must meet the following requirements:
Make the connector actions available for the agent to invoke.
Ensure authentication is handled at the service level, not per user.
Pass conversation context into the connector when it is called.
Surface the returned data of the connector to the user in the conversation.
You need to configure the agent and the connector action to meet the requirements.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Add the connector as a tool in the agent.

Why it fits: Before an agent can explore, identify, or execute specific actions or operations from a connector, the connector itself must be formally registered into the conversational runtime ecosystem by declaring it as an active **agent tool**.

2. Create or reuse a connection for the connector.

Why it fits: Connections handle the security boundaries and identity layers for external services in the Power Platform. Setting up a dedicated connection credentials profile allows the connector to run under a unified service account tier rather than prompting each unique end-user to log in individually.

3. Map topic variables to the tool inputs.

Why it fits: In order for the connector action to run using data gathered during the active conversation (such as an order ID, name, or date), you bind those dynamic, in-memory **topic variables** directly to the designated input slots expected by the tool payload.

4. Parse the JSON response and assign it to a topic variable.

Why it fits: Raw API payloads return unstructured strings or JSON records that cannot be shown to a user directly. **Parsing the JSON** maps these fields into strongly-typed conversational variables, which can then be formatted into natural text nodes or cards to display to the user.

Question: 42 CertyIQ

DRAG DROP -
A company has an Azure AI Search vector index that contains enterprise content and wants to use it as a knowledge source for an agent in Copilot Studio.
To meet business needs, the knowledge source must meet the following requirements:
The agent must connect to the Azure AI Search service.
The agent must use a supported authentication method for the connection.
The agent must ensure answers are grounded using the specified Azure AI Search index.
You need to connect to Azure AI Search as a knowledge source for the agent.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

Azure AI Search knowledge source connection steps
Configure the authentication method and provide the required connection details.
Associate the configured knowledge source with the agent.
Create a new Azure AI Search connection.
Initiate the process to add a knowledge source to the agent.
Specify the Azure AI Search index to use for grounding.

Answer Area	
1	Initiate the process to add a knowledge source to the agent.
2	Create a new Azure AI Search connection.
3	Configure the authentication method and provide the required connection details.
4	Specify the Azure AI Search index to use for grounding.
5	Associate the configured knowledge source with the agent.

Answer:

Azure AI Search knowledge source connection steps
Configure the authentication method and provide the required connection details.
Associate the configured knowledge source with the agent.
Create a new Azure AI Search connection.
Initiate the process to add a knowledge source to the agent.
Specify the Azure AI Search index to use for grounding.

Answer Area	
1	Initiate the process to add a knowledge source to the agent.
2	Create a new Azure AI Search connection.
3	Configure the authentication method and provide the required connection details.
4	Specify the Azure AI Search index to use for grounding.
5	Associate the configured knowledge source with the agent.

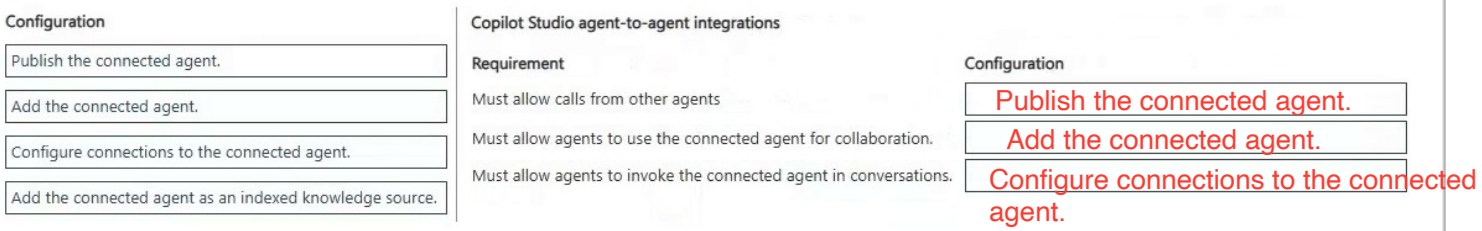
Explanation:

1. **Initiate the process**: Within Microsoft Copilot Studio, navigate to the agent's **Knowledge** settings tab and select the option to introduce a new knowledge repository to launch the configuration wizard.
2. **Create a connection**: Select the **Azure AI Search** connector option from the marketplace integration catalog to initiate a fresh connection interface.
3. **Configure authentication**: Input the baseline structural fields required for the secure handshake, including the service **Endpoint URL** and selecting the appropriate credentials protocol (such as an API Key or Service Principal).
4. **Specify the index**: Choose or type the targeted **Search Index** container name where your documents have been compiled so the agent points to the correct partition.
5. **Associate with the agent**: Conclude the setup by finalizing the entry to bind and register the validated knowledge pipeline into the conversational agent's active runtime.

Question: 43

CertyIQ

DRAG DROP -
A company uses a primary agent in Copilot Studio to handle user intake.
For some requests, the company wants the primary agent to delegate work to another Copilot Studio agent that already exists.
To meet the business needs, the solution must allow agents to perform the following on the connected agent:
Must allow calls from other agents.
Must allow agents to use the connected agent for collaboration.
Must allow agents to invoke the connected agent in conversations.
You need to integrate the existing Copilot Studio agent with the connected agent.
Which setting should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer: Authorizing Runtime Invocation (Configure connections to the connected agent): To invoke the connected agent during live conversations, the primary agent must establish and configure valid connections (passing user/ authentication context) between the calling agent and the target connected agent

Explanation:

1. Publish the connected agent.

Why it fits: Before any external service, application, or secondary conversational agent can discover, connect to, or successfully pass payload calls to an agent, that target agent's live schema and runtime code must be compiled and deployed to the cloud by executing a **Publish** operation.

2. Add the connected agent.

Why it fits: To enable a multi-agent architectural design pattern where a primary orchestrator can collaborate with and pass tasks to a subordinate sub-agent, the primary agent's workspace configuration must explicitly reference that target agent. You fulfill this by selecting **Add the connected agent** within the workspace.

3. Configure connections to the connected agent.

Why it fits: Allowing one agent to safely invoke another during live production chat sessions requires satisfying security policies and endpoint handshakes. Setting up the authentication layer, trust relationship, and routing rules is done under **Configure connections to the connected agent.**

Question: 44 CertyIQ

A company uses multiple Copilot Studio agents that perform specialized tasks.
The company needs to enable one agent to directly delegate and invoke another agent within Copilot Studio by using a standard Copilot Studio interface.
You need to configure collaboration among agents by using Copilot Studio.
What should you do?

A. Add the target agent as a Microsoft Fabric Data Agent.
B. Provide A2A endpoint information for the target agent.
C. Upload documentation about the target agent for referencing.
D. Add the target agent as an indexed knowledge source.

Answer: B

Explanation:

B. Provide A2A endpoint information for the target agent.

The Agent-to-Agent (A2A) Protocol: Copilot Studio utilizes the native Agent2Agent (A2A) protocol as an open standard to facilitate multi-agent communication, orchestration, and task delegation.
Onboarding via the Interface: To bind a target agent using this framework, you navigate to the **Agents** tab of the orchestrator agent, select **Add an agent**, choose **Agent2Agent**, and then provide the unique **Endpoint URL** of the target agent. This registers the second agent as an executable tool that the main agent can dynamically invoke during multi-turn user conversations.

Question: 45 CertyIQ

A company uses an agent in Copilot Studio that produces structured responses using a reusable instruction template. The responses follow the required tone and format. During a compliance review, the team discovers that the responses were generated using a model that is not on the approved list.
The company requires that the reusable instruction template continues to be used and the model used for prompt-driven responses meets governance requirements.
The solution must ensure that prompt-driven responses are generated using an approved model.
You need to configure the solution that meets the requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Publish the agent so that the model used by prompt-driven responses is updated automatically.

B. Enable citations so generated responses can be traced and comply with governance requirements.

C. Add a line to the instruction template that requires the agent to use only approved models when generating responses.

D. Configure Azure AI Search as the grounding source so the approved model requirement is enforced during response generation.

E. Ensure the reusable instruction template is the one applied to the agent that generates the structured response.

Answer: BE

Explanation:

Template Continuity (E): Since the company explicitly requires that the reusable instruction template continues to be used to dictate tone and format, you must verify that this specific instruction template remains bound to the node or topic where the agent generates the structured response.
Governance and Compliance (B): Enabling citations directly satisfies the needs of the compliance review team. By forcing the agent to output explicit references and citations for the data it synthesizes, compliance officers can trace exactly where the model sourced its information, ensuring the prompt-driven answers align with organizational compliance and data governance policies.

Question: 46 CertyIQ

A team has an agent in Copilot Studio that handles user intake. For some requests, the team wants the agent to delegate work to a Microsoft Foundry agent that performs specialized processing. The Foundry agent already exists.
To meet the business needs, the solution must meet the following requirements:
The primary agent must be able to call the Foundry agent.
The external agent connection must reference the correct Foundry project and agent.
You need to integrate the primary agent with the Foundry agent.
Which two actions should you take? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Add an external agent.

B. Configure the Copilot Studio endpoint URL.

C. Configure the project endpoint URL.

D. Add an indexed knowledge source.

E. Publish the Copilot Studio agent.

Answer: AC

Explanation:

Add an external agent (A): To connect your primary user-facing Copilot Studio agent to an external processor like Microsoft Foundry, you navigate to your main agent's configuration, open the **Agents** page, select **Add an agent**, and under **Connect to an external agent**, choose **Microsoft Foundry**.

Project endpoint URL (C): During the connection setup process within Copilot Studio, to securely reference and communicate with the specific background Foundry asset registry, you are explicitly prompted to create a new connection by providing the target **Foundry project endpoint URL** along with the specific Foundry Agent ID.

Question: 47

CertyIQ

A company must enable an agent in Copilot Studio to execute operations exposed by an external MCP server.
The MCP server publishes callable tools and requires authentication.
The agent must be able to invoke MCP tools during conversations.
You need to configure MCP tools so the agent can execute MCP tool calls.
What should you do?

A. Upload documentation about the MCP server so the agent can answer questions about it.

B. Enable generative answers so the model can automatically invoke MCP tools.

C. Publish the agent and rely on default settings for tool availability.

D. Add the MCP server to the agent's tool configuration.

Answer: D

Explanation:

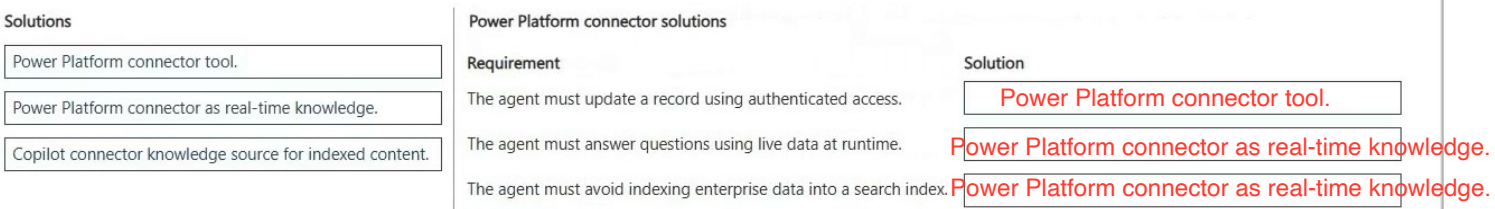
D. Add the MCP server to the agent's tool configuration.

Native Integration Interface: To connect an external MCP server to Copilot Studio, you must explicitly register it under the agent's configuration settings. You do this by navigating to the **Agents** configuration tab, choosing **Tools**, and selecting the option to add an existing MCP server.

Question: 48

CertyIQ

DRAG DROP -
A company needs an agent to answer questions using live enterprise records and to execute a limited set of operations in the same system.
To meet the business needs, the agent must meet the following requirements:
The agent must update a record using authenticated access.
The agent must answer questions using live data at runtime.
The agent must avoid indexing enterprise data into a search index.
You need to select the correct Power Platform connector approach for each requirement.
Which option should you use for each requirement? To answer, move the appropriate solutions to the correct requirements. You may use each solution once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:

Explanation:

1. Power Platform connector tool.

Why it fits: In Copilot Studio, when an agent needs to perform an active operation or mutation (such as creating, updating, or deleting a record), it utilizes a **tool** (formerly known as an action or plugin). These tools execute operations via standard Power Platform connectors under authenticated user context or predefined API connections.

2. Power Platform connector as real-time knowledge.

Why it fits: When an agent needs to provide answers using the most current, up-to-the-minute live data directly at the moment a user asks a question, the connector is added to the agent's **knowledge base as a real-time source**. The agent issues queries directly to the source system at runtime instead of relying on cached or scheduled data.

3. Power Platform connector as real-time knowledge.

Why it fits: Choosing a **real-time knowledge source** bypasses the ingest-and-index pipeline. Instead of scraping and copying enterprise data into an external storage database or a search service (which is what a knowledge source for indexed content does), it pulls data directly on-the-fly and drops it from memory once the response is formulated, preventing data from being indexed.

Question: 49

CertyIQ

A company uses an agent in Copilot Studio to coordinate user interactions.
The company wants the agent to delegate requests to a Microsoft Fabric Data Agent that accesses governed business data.
The agent must retrieve structured analytical data while enforcing data security and scope controls.
You need to select the integration approach that meets the requirements.
What should you use?

A. A Microsoft Fabric Data Agent and bind it to the model.
B. A REST API action that queries Fabric tables directly.
C. A Power Automate flow and call it from a topic.
D. A Copilot connector knowledge source that indexes Fabric data.

Answer: A

Explanation:

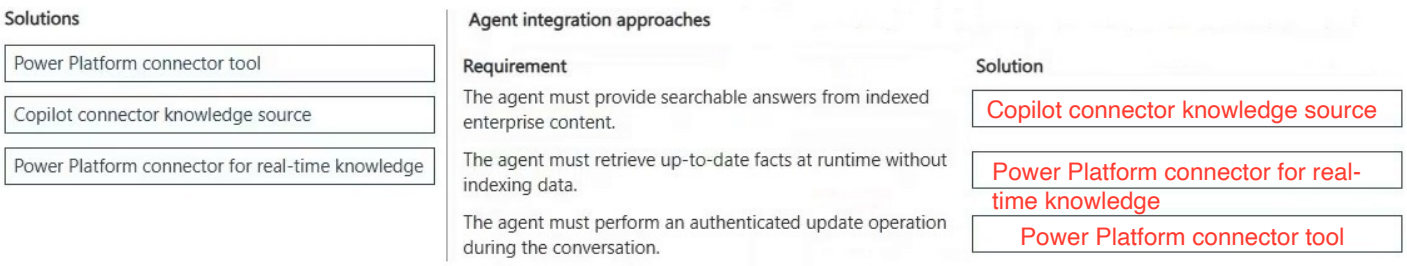
A. A Microsoft Fabric Data Agent and bind it to the model.

Agentic Collaboration: One of the primary deployment mechanisms for Microsoft Fabric Data Agents is to add them directly to a custom AI agent in **Microsoft Copilot Studio** as a **connected agent**. This establishes a native multi-agent architecture via Model Context Protocols (MCP).
Security & Scope Enforcement: Instead of querying database tables directly, binding a published Fabric Data Agent to your orchestration model allows Copilot Studio to delegate complex data queries to Fabric. This delegation natively respects and enforces all pre-configured enterprise data governance policies, permissions, and security scope definitions (such as row-level or column-level security) tied to the user's data context.

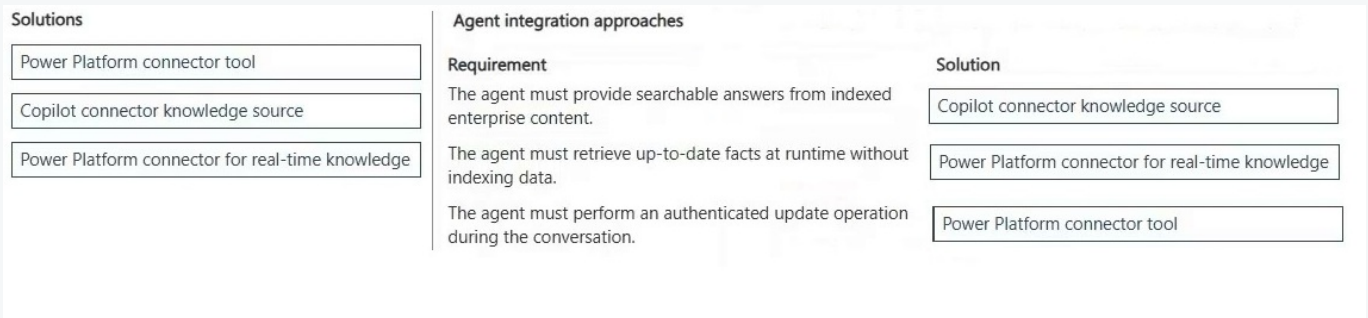
Question: 50

CertyIQ

DRAG DROP -
A company is building an agent in Copilot Studio that must answer questions using enterprise systems and must also perform authenticated operations.
To meet the business needs, the agent must meet the following requirements.
The agent must provide searchable answers from indexed enterprise content.
The agent must retrieve up-to-date facts at runtime without indexing data.
The agent must perform an authenticated update operation during the conversation.
You need to select the correct integration approach for each requirement.
Which option should you use for each requirement? To answer, move the appropriate solutions to the correct requirements. You may use each solution once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Copilot connector knowledge source

Why it fits: When enterprise content needs to be crawled, parsed, and optimized for broad semantic search across an entire repository, it is configured as an **indexed knowledge source**. The content is ingested into a search index ahead of time to allow fast retrieval of complex answers.

2. Power Platform connector for real-time knowledge

Why it fits: If data cannot be cached or stored in an external search index due to security restrictions or a need for instant freshness, using a connector as a **real-time knowledge source** queries the live external system directly at runtime. It pulls the necessary facts on the fly without persisting them to an index.

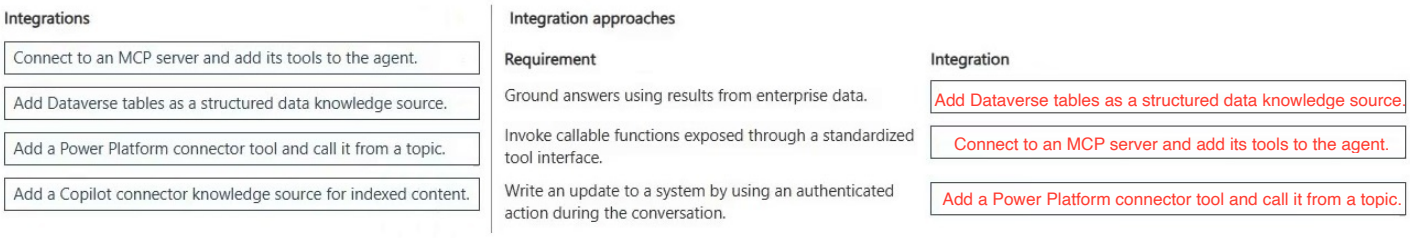
3. Power Platform connector tool

Why it fits: Any action that alters data or executes a transactional business operation (such as updating a status field, changing a record, or executing a workflow) is handled as a **connector tool** (action/plugin). It uses authenticated context to safely modify data rather than just retrieving informational content.

Question: 51

CertyIQ

DRAG DROP -
A company is extending agents in Copilot Studio with both external capabilities and enterprise content access.
Some user requests require grounded answers, while other requests require executing actions.
To meet the business needs, the agent must meet the following requirements:
Ground answers using results from enterprise data.
Invoke callable functions exposed through a standardized tool interface.
Write an update to a system by using an authenticated action during the conversation.
You need to integrate the correct approach to meet the requirements.
What should you integrate for each requirement? To answer, move the appropriate integrations to the correct requirements. You may use each integration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:

Explanation:

1. Add Dataverse tables as a structured data knowledge source

Why it fits: Microsoft Dataverse holds the core transactional and organizational data of an enterprise. By adding these tables directly as a structured knowledge source, the conversational agent can natively search, retrieve, and ground its generative responses using authoritative enterprise database records.

2. Connect to an MCP server and add its tools to the agent

Why it fits: The **Model Context Protocol (MCP)** is an open, modern standard engineered specifically to act as a universal, standardized interface between AI models and external development tools or API systems. Connecting to an MCP server allows the agent to safely discover and invoke exposed functions using a uniform schema.

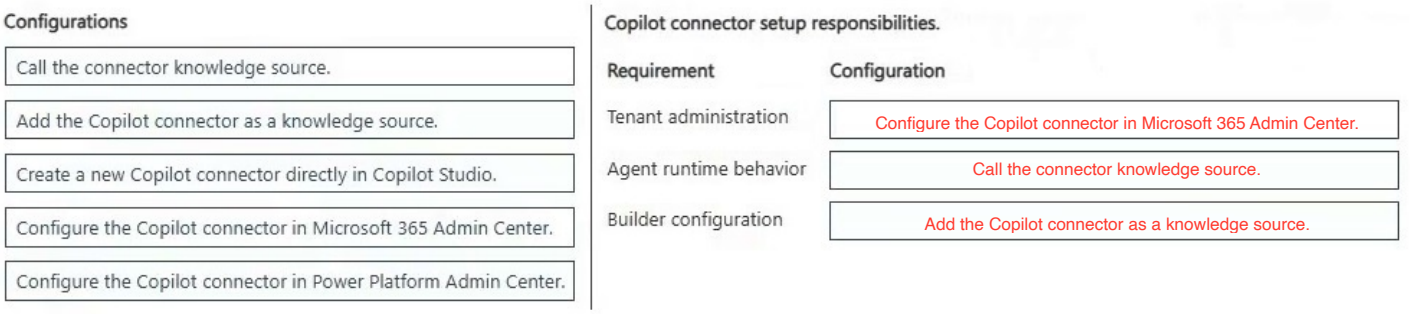
3. Add a Power Platform connector tool and call it from a topic

Why it fits: When a conversation demands an active side-effect or state change in an external system (such as writing or modifying a record), the agent triggers a custom **connector tool**. Triggering this tool from a specific conversation topic ensures that the operation executes with proper user authentication and transactional integrity.

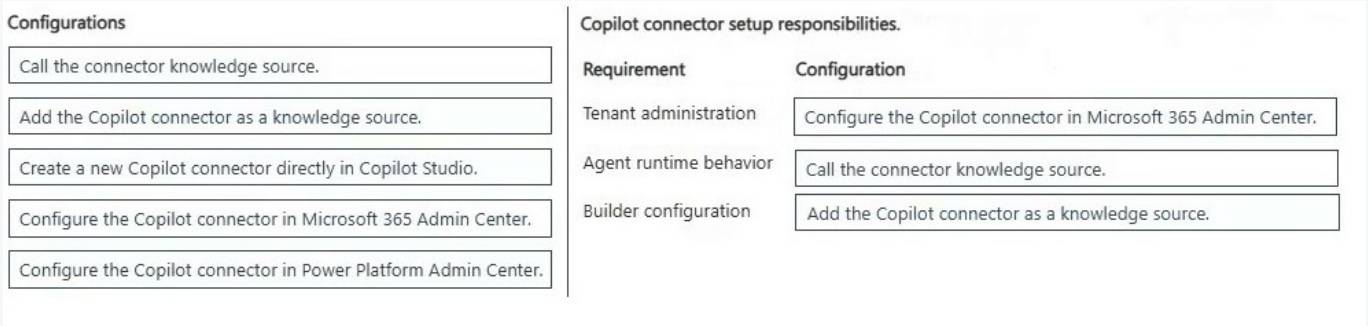
Question: 52

CertyIQ

DRAG DROP -
A company plans to use Copilot connectors to ground answers from a third-party system for an agent in Copilot Studio.
To meet the business needs, responsibilities for configuring and using Copilot connectors must be separated, based on the following requirements:
For tenant administration, the connector must be available at the tenant level.
For agent runtime behavior, answers must be grounded from authorized sources.
For builder configuration, you must add the connector to the agent.
You need to determine how Copilot connector set-up responsibilities are handled across tenant, runtime, and builder stages.
What should you configure for each requirement? To answer, move the appropriate configuration to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Configure the Copilot connector in Microsoft 365 Admin Center.

Why it fits: Broad administrative operations, tenant-wide data access permissions, data ingestion management, and deployment authorization of organizational connectors into the enterprise ecosystem fall under the purview of **Tenant administration** within the Microsoft 365 Admin Center.

2. Call the connector knowledge source.

Why it fits: This represents an operational event that happens dynamically during an end-user session. When a user asks a question, the agent triggers real-time querying to retrieve context from the connected data source, defining its **Agent runtime behavior**.

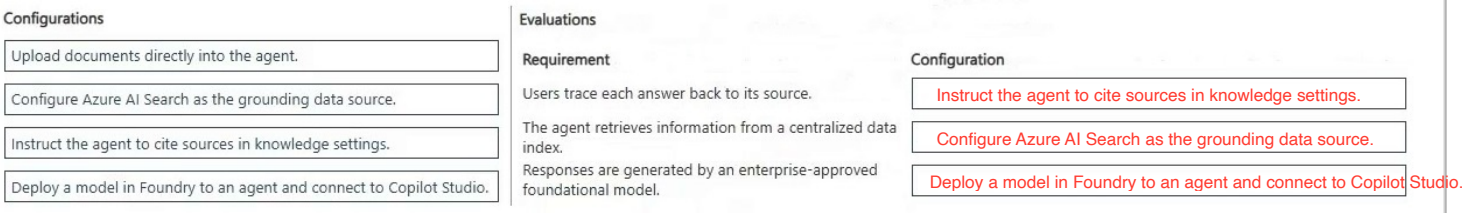
3. Add the Copilot connector as a knowledge source.

Why it fits: This is an implementation step handled by the app maker or bot author. Within Microsoft Copilot Studio, the maker links a validated data connector to the agent's knowledge repository so it can ground generative answers, constituting **Builder configuration**.

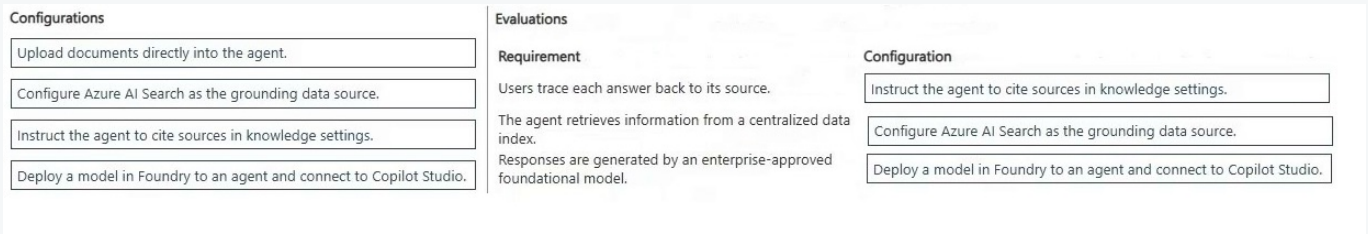
Question: 53

CertyIQ

DRAG DROP -
A company uses an agent in Copilot Studio to answer questions based on enterprise content.
The content is indexed in Azure AI Search. Responses must be generated by a Microsoft Foundry model.
To meet the business needs, the solution must meet the following requirements:
Users are able to trace each answer back to its source.
The agent retrieves information from a centralized data index.
Responses are generated by an enterprise-approved foundation model.
You need to configure generative answers for the agent.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Instruct the agent to cite sources in knowledge settings.

Why it fits: Enabling source citation forces the agent to append inline references, links, or footnotes pointing to the exact documents or snippets used to generate the output. This fulfills the capability for end-users to verify and trace facts back to their origin.

2. Configure Azure AI Search as the grounding data source.

Why it fits: Azure AI Search acts as a **centralized search index** for large enterprise data silos. Connecting it as a knowledge source allows Copilot Studio to perform advanced hybrid/vector searches over pre-indexed company assets to ground generative AI outputs.

3. Deploy a model in Foundry to an agent and connect to Copilot Studio.

Why it fits: When enterprise compliance prohibits the default model and mandates a vetted, highly tailored LLM, you deploy that specific **foundational model** inside Azure AI Foundry (formerly Azure AI Studio) and map it back into Copilot Studio as the primary model engine.

Question: 54

CertyIQ

A company uses an agent in Copilot Studio to generate structured responses for internal users.
The agent must meet the following requirements:
Consistently follow a reusable instruction template for response tone and structure.
Use an enterprise-approved foundation model for responses generated through the template.
You need to configure custom prompts so the agent uses the Microsoft Foundry model catalog.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Publish the agent to apply the Foundry model selection automatically.
B. Configure Azure AI Search as the grounding source, so the model choice is enforced.
C. Allow topic authors to define tone and structure directly in topic steps.
D. Select a model from the Foundry model catalog for the custom prompt configuration.
E. Create a custom prompt template and apply it where the agent generates the structured response.
F. Enable citations, so the model is restricted to approved outputs.

Answer: DE

Explanation:

D. Select a model from the Foundry model catalog for the custom prompt configuration.

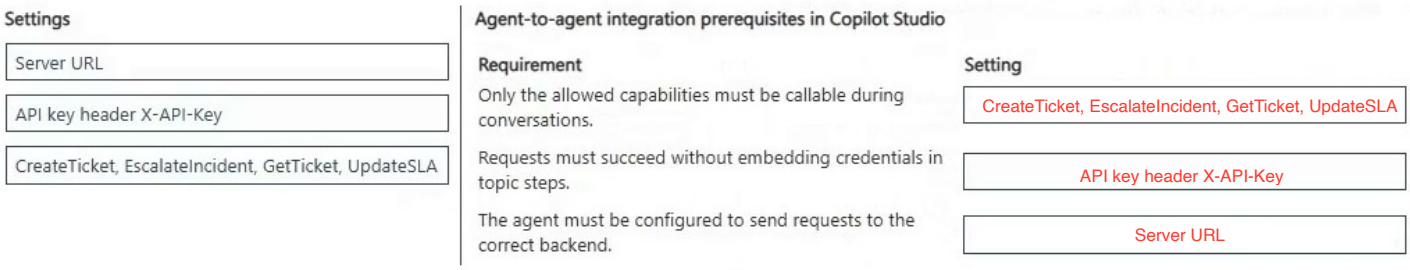
E. Create a custom prompt template and apply it where the agent generates the structured response.

Custom Prompt Template (E): Creating a reusable prompt template (using the Prompt tool in Copilot Studio) is the native method to enforce a consistent instruction format, tone, and response structure. Once crafted, this template or prompt action can be seamlessly applied across any node or topic where a structured response is required.
Selecting a Foundry Model (D): Under the Bring Your Own Model (BYOM) capability within the prompt tool configuration, makers can change the model from the default managed choices to a custom enterprise-approved option. By clicking the model selection menu, you can connect and select a specific base or fine-tuned model deployed via the **Azure AI Foundry / Microsoft Foundry model catalog**.

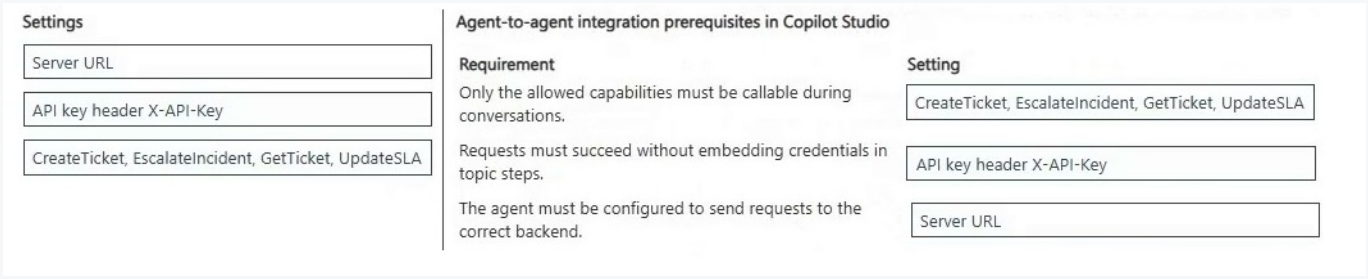
Question: 55

CertyIQ

DRAG DROP -
A company integrates an MCP service that exposes ticket-management operations.
The MCP service requires authenticated requests and exposes multiple callable operations that must be selectively governed for agent use.
To meet the business needs, the MCP configuration must meet the following requirements:
Only the allowed capabilities must be callable during conversations.
Requests must succeed without embedding credentials in topic steps.
The agent must be configured to send requests to the correct backend.
You need to configure MCP tools for the agent.
Which setting should you configure for each requirement? To answer, move the appropriate settings to the correct requirements. You may use each setting once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. CreateTicket, EscalateIncident, GetTicket, UpdateSLA

Why it fits: This comma-separated list defines specific operations or functions. By explicitly declaring this restricted set of endpoints or tool paths during configuration, you limit the agent's scope, ensuring it can only invoke those specific capabilities.

2. API key header X-API-Key

Why it fits: To comply with security best practices and keep credentials out of cleartext dialog fields, authentication parameters are abstracted to the tool configuration level. Setting up the token format here handles credentials globally for the connection.

3. Server URL

Why it fits: The **Server URL** represents the base network path (endpoint) where the target backend service or

API server resides, ensuring that outbound API requests reach the intended destination system.

Question: 56 CertyIQ

An agent in Copilot Studio must complete tasks by automating interactions with a legacy desktop application that does not use APIs.
During testing, the builder must confirm that the automation runs in a supported execution context and can inspect how each automation step executes during conversations.
To meet the business needs, the solution must:
Execute UI automation in a supported, platform-managed environment.
Provide visibility into how automation steps run during agent conversations.
You need to configure the agent based on the requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Publish the agent so automation settings are applied at runtime.
B. Inspect automation execution details by using the agent runtime view.
C. Enable computer use for the agent so UI automation runs in the supported execution environment.
D. Add the desktop application as a knowledge source so the agent can reference its UI structure.
E. Configure a custom connector to simulate UI interactions through REST calls.

Answer: BC

Explanation:

B. Inspect automation execution details by using the agent runtime view.

C. Enable computer use for the agent so UI automation runs in the supported execution environment.

Enable computer use (C): The **Computer use** tool in Copilot Studio is explicitly designed to solve the problem of automating legacy applications without native APIs. Enabling this allows the agent to interact dynamically with desktop or web screens (typing, clicking, and viewing UI layouts) based on natural language guidelines.
Agent Runtime View (B): When executing or diagnosing a computer-use-enabled agent, the testing pane or runtime view provides a live session feedback log. This allows makers to look at step-by-step screenshots, action latency, function matching logs, and full execution trace logs to audit how the agent interacted with the legacy UI.

Question: 57 CertyIQ

DRAG DROP -
A company configures an agent flow that interacts with multiple systems to complete user requests.
The company must integrate external services, retrieve data, and notify users as part of the same automated flow run. To support this scenario, the agent flow must be able to do the following:
Call an external API.
Retrieve stored data from a supported internal data source.
Send a Teams message.
You need to select a connector or action type for each requirement.

What should you use? To answer, move the appropriate connectors or actions to the correct requirements. You may use each connector or action once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Connector or action

- Trigger a notification.
- Make an HTTPS request.
- Add a standard connector.

Connectors and actions

Requirement	Connector or action
Call an external API.	Make an HTTPS request
Retrieve stored data from a supported internal data source.	Add a standard connector.
Send a Teams message.	Trigger a notification.

Answer:

Connector or action

- Trigger a notification.
- Make an HTTPS request.
- Add a standard connector.

Connectors and actions

Requirement	Connector or action
Call an external API.	Make an HTTPS request.
Retrieve stored data from a supported internal data source.	Add a standard connector.
Send a Teams message.	Trigger a notification.

Explanation:

1. Make an HTTPS request.

Why it fits: When an agent needs to communicate directly with an external API endpoint that doesn't have a pre-built connector wrapper, making a raw HTTP/HTTPS request allows you to define the exact method (GET, POST, etc.), headers, and payload structure.

2. Add a standard connector.

Why it fits: Out-of-the-box Microsoft ecosystem data repositories (such as SharePoint, SQL Server, Excel, or OneDrive) are integrated by utilizing the platform's library of **standard connectors**. Adding these allows the agent to read and query stored data securely without writing custom integration code.

3. Trigger a notification.

Why it fits: Sending an asynchronous update or posting a dynamic message directly into a Microsoft Teams chat or channel serves the specific functional goal of **triggering an outbound system notification** to alert a user or a team.

Question: 58

CertyIQ

An agent in Copilot Studio must retrieve real-time data from an external system that exposes a REST API.
The API returns structured JSON data and requires authentication.
The agent must call the API during conversations to fulfill user requests.
The agent must retrieve real-time data from the external system by calling an authenticated REST API during a conversation.

The solution must use a supported mechanism that executes the REST API call and the call must be explicitly configured to run at runtime.
You need to determine which approaches add the REST API to the agent.
Which two solutions meet the goal? Each correct answer is a complete solution.
NOTE: Each correct selection is worth one point.

A. Configure an HTTP action and invoke it from a topic during the conversation.
B. Create a topic variable and use a Formula to parse the API endpoint string.
C. Add the REST API as a tool and call it either by using a topic or agent instruction.
D. Add the REST API as a knowledge source so the agent can query it during conversations.
E. Enable generative answers and rely on the model to fetch the REST API data automatically.
F. Create a global variable to store the API URL and call it using a Redirect node.

Answer: AC

Explanation:

A. Configure an HTTP action and invoke it from a topic during the conversation.
C. Add the REST API as a tool and call it either by using a topic or agent instruction.

HTTP Action (A): Copilot Studio includes a native HTTP Request node that can be embedded directly into a topic workflow. This node allows you to explicitly provide the target endpoint URL, select the appropriate method (GET, POST, etc.), append authentication headers (such as Authorization: Bearer <token>), and parse the returning structured JSON response into a runtime variable.
REST API as a Tool / Action (C): Under the agent customization capabilities, you can extend an agent by adding actions or tools using a REST API definition (such as uploading an OpenAPI/Swagger file specification). Once registered, this tool can either be explicitly invoked via a topic action node or left for the agent's orchestration engine to dynamically call based on user instructions and trigger parameters.

Question: 59 CertyIQ

A company must enable an agent in Copilot Studio to execute operations exposed by an external MCP server.
The MCP server publishes callable tools and requires authentication.
The agent must be able to invoke MCP tools during conversations.
You need to configure MCP tools so the agent can execute MCP tool calls.
What should you do?

A. Enable generative answers so the model can automatically invoke MCP tools.
B. Add a Power Platform connector as real-time knowledge to replace MCP tools.
C. Add the MCP server output as an indexed knowledge source.
D. Configure authentication for the MCP server connection used by the agent.

Answer: D

Explanation:

D. Configure authentication for the MCP server connection used by the agent.

Security Requirements: When connecting Copilot Studio to an external MCP server via the MCP onboarding wizard, configuring the target authentication mechanism (such as providing an API key or setting up the **OAuth 2.0 Authorization Code Flow**) is a mandatory configuration requirement.
Runtime Execution: This authentication configuration behind the scene maps to a backend Power Platform custom connector structure, enabling the agent's orchestrator to pass valid security credentials securely during runtime conversational tool calls without exposing plain text configurations.

Question: 60

CertyIQ

DRAG DROP -
An agent in Copilot Studio must retrieve real-time data from an external system that exposes a REST API. The API returns structured JSON responses and requires authentication.
To meet the business needs, the solution must meet the following requirements:
The agent calls a REST API during a conversation.
The REST API call runs with proper authentication.
The REST API response is available to the topic at runtime.
You need to configure the REST API to meet the requirements.
What should you configure for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at au. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Store the API response in a topic variable.
- Add the API as an indexed knowledge source.
- Build an authenticated connection for the API.
- Configure a REST action and call it from a topic.

REST API

Requirement	Configuration
Agent calls a REST API during a conversation.	Configure a REST action and call it from a topic.
REST API call runs with proper authentication.	Build an authenticated connection for the API.
REST API response is available to the topic at runtime.	Store the API response in a topic variable.

Answer:

Configurations

- Store the API response in a topic variable.
- Add the API as an indexed knowledge source.
- Build an authenticated connection for the API.
- Configure a REST action and call it from a topic.

REST API

Requirement	Configuration
Agent calls a REST API during a conversation.	Configure a REST action and call it from a topic.
REST API call runs with proper authentication.	Build an authenticated connection for the API.
REST API response is available to the topic at runtime.	Store the API response in a topic variable.

Explanation:

1. Configure a REST action and call it from a topic.

Why it fits: To invoke an external REST API directly during a guided conversational flow, you create a custom action or tool detailing the HTTP method and endpoint. This action is then added as a node inside the conversational topic canvas to execute the API call.

2. Build an authenticated connection for the API.

Why it fits: Ensuring that the API executes securely requires setting up proper credentials (such as API keys, OAuth, or Bearer tokens). Building an authenticated connection handles these security layer mappings so the endpoint doesn't reject the incoming request.

3. Store the API response in a topic variable.

Why it fits: Once the REST API returns a payload (such as JSON data), the agent needs to save that information to use it in subsequent dialog nodes. Saving the outputs to a **topic variable** allows the conversation to access, display, or filter the returned data fields at runtime.

Question: 61 CertyIQ

A company is configuring generative answers for an agent in Copilot Studio agent.
The agent must use Azure AI Search to retrieve enterprise content, and all responses must be generated by a Microsoft Foundry model.
The agent sometimes generates answers that are not grounded in Azure AI Search content. The company requires that the agent only generate responses when relevant indexed content is retrieved.
You need to configure the agent so that its generative answer behavior meets the organization's requirements.
What should you do?

A. Use Azure AI Search as a tailback source and enable answer expansion.
B. Upload documents into the agent and disable fallback responses.
C. Require grounding for generative answers and use Azure AI Search as the retrieval source.
D. Enable citations and configure a public model endpoint.
E. Enable conversational boosting and select a Foundry-approved model.

Answer: C

Explanation:

C. Require grounding for generative answers and use Azure AI Search as the retrieval source.

Enforcing Strict Grounding: In Copilot Studio, turning off the "Allow ungrounded responses" feature or selecting options that require grounding for generative answers forces the system to only output responses when it successfully retrieves relevant, matching text blocks from its indexes. If the question is outside the scope of the data found, the agent will gracefully fail over or state that it cannot find the information, eliminating ungrounded fabrications (hallucinations).
Retrieval Mapping: Explicitly coupling this strict grounding policy with **Azure AI Search** directs the orchestration engine to use your designated indexed enterprise document catalog as the exclusive, authorized source of truth for the Microsoft Foundry model to summarize.

Question: 62 CertyIQ

DRAG DROP -
A company wants to extend an agent in Copilot Studio by calling an external REST API to retrieve and update data.
To meet the business needs, the REST API must be added following a correct dependency order.

You need to add a REST API to the agent so it can be invoked during conversations.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

REST API	Answer Area	
Add the REST API tool.	1	Add the REST API tool.
Test the API configuration.	2	Configure the API endpoint details.
Configure authentication for the API.	3	Configure authentication for the API.
Configure the API endpoint details.	4	Test the API configuration.

Answer:

REST API	Answer Area	
Add the REST API tool.	1	Add the REST API tool.
Test the API configuration.	2	Configure the API endpoint details.
Configure authentication for the API.	3	Configure authentication for the API.
Configure the API endpoint details.	4	Test the API configuration.

Explanation:

1. **Add the REST API tool**: You begin by navigating to the agent's tools section and initiating the setup process by selecting **New tool > REST API**.
2. **Configure the API endpoint details**: Next, you supply the API specification layout (typically by uploading an **OpenAPI/Swagger JSON file**) to define the paths, methods, input parameters, and endpoint routes.
3. **Configure authentication for the API**: Once the structural details of the endpoint are established, you configure the necessary security protocols, such as API keys, basic authentication, or OAuth 2.0 flow parameters.
4. **Test the API configuration**: Finally, you validate the end-to-end integration by launching a test session to confirm that the agent can successfully reach the API endpoint, handle credentials properly, and receive valid responses.

Question: 63 CertyIQ

DRAG DROP -
A company needs an agent in Copilot Studio to complete tasks by interacting with a legacy desktop application that does NOT have APIs.
To meet the business needs, the solution must meet the following requirements:
The agent must interact with the application's UI.
The execution must run on supported infrastructure.
The builder must be able to observe how the tool runs during execution.
You need to configure the agent based on the requirements.
Which setting should you configure for each requirement? To answer, move the appropriate settings to the correct requirements. You may use each setting once, more than once, or not at all. You may need to move the split bar

between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Settings

- Enable Computer use tool for the agent.
- Monitor run details in the agent runtime view.
- Use a platform-hosted execution environment.

Agent-to-agent integration prerequisites in Copilot Studio

Requirement	Setting
Agent interacts with the application's UI.	Enable Computer use tool for the agent.
Execution runs on supported infrastructure.	Use a platform-hosted execution environment.
Builder observes how the tool runs during execution.	Monitor run details in the agent runtime view

Answer:

Settings

- Enable Computer use tool for the agent.
- Monitor run details in the agent runtime view.
- Use a platform-hosted execution environment.

Agent-to-agent integration prerequisites in Copilot Studio

Requirement	Setting
Agent interacts with the application's UI.	Enable Computer use tool for the agent.
Execution runs on supported infrastructure.	Use a platform-hosted execution environment.
Builder observes how the tool runs during execution.	Monitor run details in the agent runtime view.

Explanation:

1. Enable Computer use tool for the agent.

Why it fits: The **Computer Use** capability allows AI agents to read a digital display, move a cursor, click buttons, and type text just like a human operator. Enabling this tool is a prerequisite whenever an agent needs to programmatically navigate and interact directly with an application's user interface.

2. Use a platform-hosted execution environment.

Why it fits: Operating a virtual desktop canvas or interacting with desktop elements requires secure, sandboxed compute resources. A **platform-hosted execution environment** ensures that these automation runs occur inside isolated, compliant, and pre-configured Microsoft-managed secure infrastructure.

3. Monitor run details in the agent runtime view.

Why it fits: When diagnosing or observing tool behavior, builders utilize live telemetry screens. The **agent runtime view** provides real-time access to operational sequences, step-by-step logs, and live execution details as the agent triggers its configured tools.

Question: 64

CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A company is configuring an agent in Copilot Studio that uses a generative answers node inside multiple topics.
The company requires responses to meet the following requirements:
Must use an executive summary format.
Must apply only within a specific topic.
Must continue using configured knowledge sources.
Must comply with connector security controls.

You need to configure a custom prompt to meet the formatting requirements.
Solution: Add instructions in the custom prompt tool to restrict connector access during runtime.
Does the solution meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

Wrong Tool for Security: The custom prompt tool is designed to provide natural language instructions that guide how a foundation model structures, formats, and sets the tone of its text output (e.g., enforcing an "executive summary format"). Writing text instructions in a prompt to try to programmatically restrict backend network access or data connectors at runtime is ineffective and insecure.
How Security is Handled: Connector security controls and data access boundaries are strictly enforced via the environment's security layer, authentication settings, user roles, and data loss prevention (DLP) policies — not by instructing the LLM via prompt engineering.

Question: 65

CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A company is configuring an agent in Copilot Studio that uses a generative answers node inside multiple topics.
The company requires responses to meet the following requirements:
Must use an executive summary format.
Must apply only within a specific topic.
Must continue using configured knowledge sources.
Must comply with connector security controls.
You need to configure a custom prompt to meet the formatting requirements.
Solution: Create a custom prompt tool in the specific topic that instructs the model to respond using an executive-summary structure.
Does the solution meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

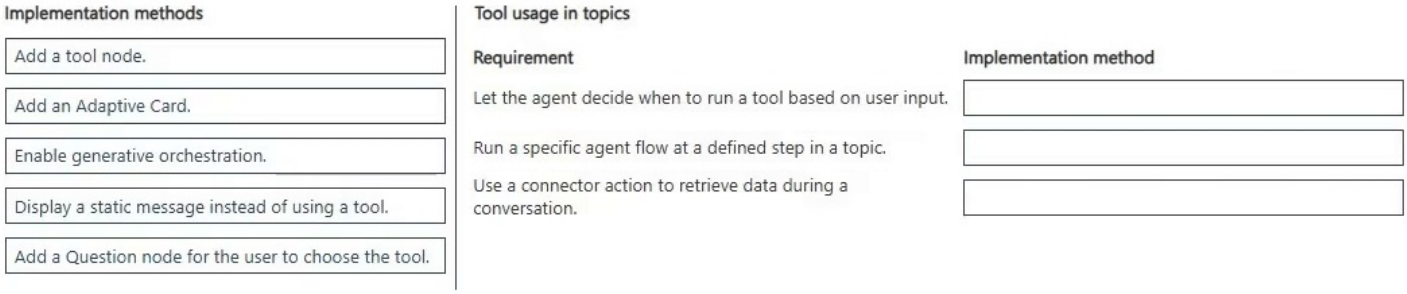
Formatting and Structure: Creating a custom prompt tool and giving it natural language instructions (e.g., "Respond using an executive-summary structure") perfectly satisfies the requirement to alter the layout and tone of the model's response.

Topic Isolation: Adding this custom prompt tool explicitly within the node structure of the specific topic ensures that the formatting constraint applies only within that exact interaction pathway, without globally altering the generative answer nodes present in the other multiple topics.
Preserving Features: It overrides the formatting behavior while naturally adhering to the existing data boundaries, configured knowledge sources, and backend connector security controls already established for that agent's runtime environment.

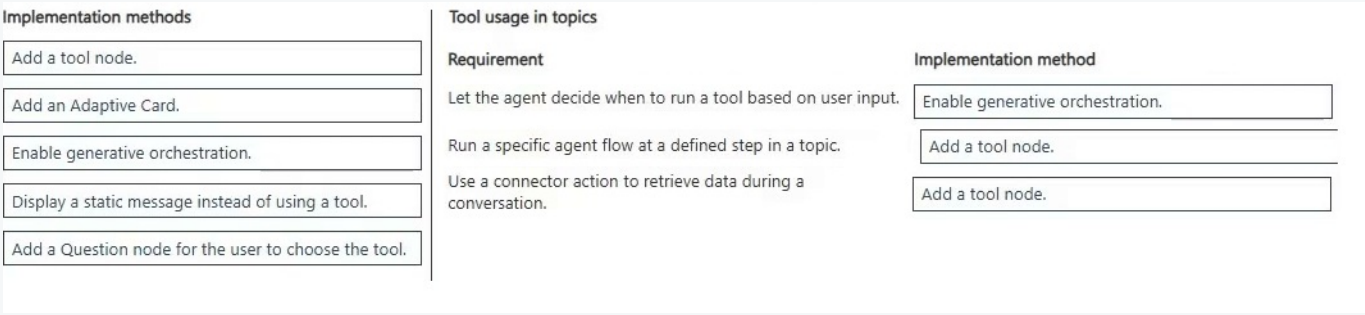
Question: 66 CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.
To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.
A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.
To start the case study -
To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.
Background -
Blue Yonder Airlines is a global carrier headquartered in Los Angeles, California, operating domestic and international flights. The company serves millions of passengers annually through its website, mobile app, and call centers. To improve customer service efficiency and reduce call center volume, Blue Yonder is deploying an AI agent in Microsoft Copilot Studio.
The agent will handle customer inquiries across multiple channels – web chat, mobile app, and Microsoft Teams (for internal support staff). It will answer questions, retrieve data from enterprise systems, and escalate to human agents when needed.
The project is led by a cross-function team:
Product manager: Defines requirements and success metrics.
Lead agent author: Designs topics, intents, and generative behavior.
Flow designers: Build agent flows and integrations.
IT/security and compliance: Oversees identity, data protection, and Responsible AI (RAI) compliance.
Current environment -
Channels -
Public website: Embedded web chat
Mobile app: In-app chatbot -
Microsoft Teams: Internal support agent access
Identity and access -
Customers: Anonymous access for general inquiries (e.g., flight status, baggage policy).
Authentication is required for personal data access (e.g., bookings, loyalty points).
Internal staff: Authenticate via Microsoft Entra ID.
Data sources -
Reservation and Ticketing System (internal): REST API, no prebuilt connector with custom enterprise database.
Flight Status and Weather APIs (external): REST APIs with API keys.
Customer Support Knowledge Base: SharePoint library with PDFs and policy documents.
Loyalty Program Data: Stored in Dynamics 365 and Dataverse.
Travel Advisory Content: Uses REST API with partner services.
Integration mechanisms -
Custom connectors must be used for internal APIs that lack prebuilt connectors.
HTTP request nodes may be used for lightweight external APIs.
Knowledge sources must be used for unstructured content.

Agent flows must be used to encapsulate reusable logic (e.g., rebooking).
Business requirements -
Omnichannel support -
Deploy the agent across web, mobile, and Teams with a consistent user experience. The Teams deployment must also support internal staff.
Self-service capabilities -
The agent must handle common inquiries such as:
Flight status -
Booking and rebooking -
Loyalty program questions -
Travel policies and baggage rules
Human escalation -
If the agent cannot resolve an issue or the user requests help, it must:
Escalate to a human agent.
Transfer the conversation transcript and relevant context.
Redact any sensitive personal data before escalation.
Knowledge integration -
The agent must use scalable methods for knowledge integration and must not rely on manually authored Q&A topics for each document.
Performance metrics -
First-contact resolution: +25%
Tier-1 call deflection: ≥20%
Response time: 90% of queries answered within 30 seconds
Accuracy: ≥95% for known FAQs -
CSAT: ≥85% for AI-handled interactions
Technical requirements -
Platform constraints -
No custom code is permitted; only Copilot Studio's built-in tools may be used.
All backend logic must be implemented using agent flows.
Markdown must be used for formatting (e.g., bold, bullet points); HTML is not supported.
Authentication -
Sign-in is required for personal data access.
Anonymous access is allowed for general inquiries.
User identity must be used for data access; shared or builder credentials must not be used.
Compliance and security -
Power Platform DLP policies must be enforced to block unauthorized data flows.
Responsible AI content moderation filters must be enabled.
Prompt modifications must be added to enforce tone, disclaimers, and refusal behavior.
Disclaimers must be applied consistently across all generative responses. Manual edits to individual topics must be avoided.
Monitoring and maintenance -
All conversations and actions must be logged for auditing.
Weekly reviews of transcripts and metrics must be conducted.
Issues and constraints -
API rate limits: External APIs (e.g., flight status) have usage limits. Agent flows must handle retries and caching to avoid exceeding quotas.
Knowledge base limits: Copilot Studio has limits on the number and size of indexed documents. Large files must be split or summarized.
Generative answer risks: Generative responses must be constrained to avoid policy violations. Prompt modifications and filters must be used to enforce tone, safety, and compliance.
User input variability: Users phrase questions in diverse ways. Topics must include varied trigger phrases and fallback handling.
Authentication UX: The agent must clearly explain when sign-in is required and handle transitions smoothly across channels.
Problem statement -
Blue Yonder Airlines must deploy a secure, scalable, and policy-compliant AI agent using Microsoft Copilot Studio. The agent must deliver accurate, helpful, and safe responses across multiple channels, integrate with enterprise systems, and support both anonymous and authenticated users. It must adhere to strict data protection and Responsible AI standards while improving customer service efficiency and satisfaction.
You need to implement tool usage in topics that meet the Blue Yonder design and governance requirements.
Which implementation method should you use for each requirement? To answer, move the appropriate implementation methods to the correct requirements. You may use each implementation method once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

1. Enable generative orchestration.

Why it fits: Generative orchestration (also known as dynamic chaining) allows the conversational agent to use AI to dynamically select and execute the most appropriate tool or topic based on the user's intent, rather than following a rigidly pre-authored conversation path.

2. Add a tool node.

Why it fits: If you need an action or business process to trigger at a precise, deterministic moment within a custom conversation path, you manually insert a tool node directly into that specific step of the topic canvas.

3. Add a tool node.

Why it fits: In Copilot Studio's unified authoring canvas, external operations — such as calling Power Platform connectors to fetch data from live systems — are encapsulated as **Tools**. To execute a connector action and bring that retrieved data back into the conversation dialog, you use a tool node.

Question: 67 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A company is configuring an agent in Copilot Studio that uses a generative answers node inside multiple topics.
The company requires responses to meet the following requirements:
Must use an executive summary format.
Must apply only within a specific topic.
Must continue using configured knowledge sources.

Must comply with connector security controls.
You need to configure a custom prompt to meet the formatting requirements.
Solution: Configure the custom prompt tool at the topic level so that other topics are not affected.
Does the solution meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

Topic Isolation: One of the strict requirements is that the executive summary format must apply only within a specific topic. Configuring the custom prompt tool locally at the individual topic level ensures that the unique formatting constraints are sandboxed, meaning generative answer nodes in other topics remain completely unaffected.

Compliance & Knowledge: Applying the prompt tool at the topic level seamlessly overrides formatting behavior while continuing to inherit and honor the globally configured knowledge sources and the environment's backend data connector security controls.

Question: 68

CertyIQ

A team is developing an agent in Copilot Studio and wants to evaluate its behavior before expanding access.
The team needs a repeatable, consistent method to test the agent using predefined interactions.
You need to ensure that agent responses can be evaluated consistently across multiple test runs.
What should you do?

A. Review evaluation scores from previous production sessions.
B. Enable telemetry and analytics to monitor intent matches, fallback usage, and response frequency.
C. Define test sets of prompts and expectations.
D. Change the evaluation method to a qualitative approach.
E. Publish the agent to additional channels for broader usage.

Answer: C

Explanation:

C. Define test sets of prompts and expectations.

Microsoft Copilot Studio includes an native Agent Evaluation feature built specifically for this purpose. This capability allows development teams to create or import **predefined test sets containing specific user prompts alongside their expected outcomes or validation parameters**.

When an evaluation is executed, the platform feeds these test cases systematically to the agent, tracks how it responds, and applies automated grading methods (such as checking for exact matches, semantic similarity, or intent accuracy). This provides an objective, repeatable quality baseline before expanding copilot access to broader user groups.

Why the other options are incorrect

A & B: Telemetry, historical analytics, and production logs monitor live or past user sessions qualitatively and dynamically. They cannot act as a structured, deterministic test script to replay identical interactions repeatedly.

D: Shifting to a purely qualitative approach relies on manual, ad-hoc ad-hoc tester feedback, which reduces consistency rather than ensuring repeatable evaluation metrics across separate builds.

E: Deploying the copilot to more channels does not solve the testing problem; it simply exposes an untested agent to real users prematurely.

Question: 69

CertyIQ

DRAG DROP -
A company builds agents in Copilot Studio in a development environment.
The agents are packaged inside a solution and deployed by using Power Platform Pipelines to test and production environments.
The company has the following requirements that must be met:
Production deployments are limited to a defined set of users who are responsible for that stage.
The same solution is reused across all environments without editing its components.
The production environment always receives a locked-down version of the solution.
You need to configure the pipeline and solution behavior to meet the requirements.
What should you configure? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Deploy the solution as managed.
- Deploy the solution as unmanaged.
- Use environment variables in the solution.
- Configure stage access for the production stage.

Pipeline stage and deployment configuration

Requirement	Configuration
Limit users who can deploy to production.	Configure stage access for the production stage
Reuse the same solution across environments without modifying its components.	Use environment variables in the solution
Ensure production receives a locked-down version of the solution.	Deploy the solution as managed.

Answer:

Configurations

- Deploy the solution as managed.
- Deploy the solution as unmanaged.
- Use environment variables in the solution.
- Configure stage access for the production stage.

Pipeline stage and deployment configuration

Requirement	Configuration
Limit users who can deploy to production.	Configure stage access for the production stage.
Reuse the same solution across environments without modifying its components.	Use environment variables in the solution.
Ensure production receives a locked-down version of the solution.	Deploy the solution as managed.

Explanation:

In Power Platform Pipelines, permissions can be defined at individual pipeline tiers. By choosing to **Configure stage access for the production stage**, administrators can use role-based security assignments to ensure only authorized deployment managers or release engineers can push builds into production.

To move an identical solution package across different landscapes (such as QA and Production) without editing components directly, you must externalize environment-specific parameters. You **Use environment variables in the solution** to handle shifting credentials, base URLs, or connection parameters seamlessly during import.

Best practice dictates that a production environment must remain immutable to accidental edits or citizen-developer interference. Choosing to **Deploy the solution as managed** explicitly locks down the copilot

elements, ensuring that components cannot be directly customized or deleted in the target system.

Question: 70 CertyIQ

HOTSPOT -
You run multiple evaluation tests for an agent in Copilot Studio before expanding user access.
The tests provide the following results:
Each evaluation case is reported as meeting or missing the expected response.
Some evaluation cases fail repeatedly across several runs.
Each run displays the expected response and the response generated by the agent.
You need to provide a conclusion based on the evaluation results.
Which conclusions should you make? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Evaluation interpretation

Evaluation result	Conclusion
Evaluation case meeting or missing expected response.	Pass or fail status / Failure configuration / Production readiness
Some evaluation cases fail repeatedly.	Prompt revisions / Consistent outcomes / Intermittent outcomes
Each run displays expected response and agent response.	Deployment approval / Root cause of failures / Expected and generated responses

Answer:

Evaluation interpretation

Evaluation result	Conclusion
Evaluation case meeting or missing expected response.	Pass or fail status
Some evaluation cases fail repeatedly.	Prompt revisions
Each run displays expected response and agent response.	Expected and generated responses

Explanation:

When an evaluation runs against a test dataset, each test case checks whether the agent's output successfully matches or fails the set expectations. This determines the overall binary **Pass or fail status** of that test case.

When specific generative test cases consistently fail across separate runs, it indicates a deterministic behavior issue (such as bad grounding, loose boundaries, or off-topic tracking). To remediate these persistent failures, the development team must implement **Prompt revisions** to tighten the system-level safety instructions or custom guardrails.

The user interface metrics screen side-by-side presents the target expected baseline next to what the copilot dynamically spit out at runtime. This provides explicit visibility into the **Expected and generated responses** to allow administrators to evaluate accuracy gaps.

Question: 71 CertyIQ

An agent in Copilot Studio is evaluated using a fixed test set and an automated evaluation method.
After running the evaluation, the team observes that the same three interactions fail in every run, while all other interactions pass consistently.
You need to use the evaluation results to determine an accurate conclusion.
What should you conclude from the evaluation results?

A. The agent's overall design is flawed and requires a full redesign.
B. The failing interactions are caused by an issue in the underlying language model.
C. The outcomes isolate which portions of the agent's functional scope require remediation.
D. The agent should not be deployed.

Answer: C

Explanation:

C. The outcomes isolate which portions of the agent's functional scope require remediation.

Because the evaluation uses a **fixed test set** and the failures are completely deterministic (the same three interactions fail every single time while everything else passes consistently), the test has successfully isolated specific, localized gaps in the agent's knowledge or logic.

This indicates that the broader system architecture and general prompt design are solid. Instead of starting over or blaming the underlying LLM, the development team can precisely target and remediate the exact functional scopes (such as a missing knowledge source, a poorly phrased topic trigger, or an ungrounded system prompt instruction) causing those three specific failures.

Question: 72 CertyIQ

A team has an existing agent in Copilot Studio and must move it to a different target location without rebuilding the agent. Some builders use Copilot Studio in the web app, while others use Copilot Studio in Microsoft Teams.
You need a supported approach to move the existing agent to the target location using ALM practices. The solution must avoid rebuilding the agent.
You need to move the existing agent to the target location using a supported method.
What should you do?

A. Remove agent components directly from the solution so that export and import will succeed.

B. Duplicate the agent in the source and manually recreate the changes in the target to avoid solution dependencies.

C. Create a custom solution, add the existing agent to the solution, export the solution with the agent, and import the solution into the target environment.

D. Publish the agent to a channel and use the channel configuration as the transport mechanism to the target environment.

Answer: C

Explanation:

C. Create a custom solution, add the existing agent to the solution, export the solution with the agent, and import the solution into the target environment.

To transport a Microsoft Copilot Studio agent between environments without rebuilding or duplicating its components, you must leverage standard Power Platform **Application Lifecycle Management (ALM)** practices.

Agents created in both the web app and Microsoft Teams are fully solution-aware components. Packing the existing agent along with its subcomponents (such as topics, entities, and environment variables) into a **custom solution container** allows you to export it as a clean zip package from your source environment and import it directly into your target destination environment.

Question: 73 CertyIQ

HOTSPOT -
You are setting up an evaluation for an agent in Copilot Studio before expanding access to more users.
You must configure the evaluation so that the agent is tested by using prepared interactions rather than impromptu conversations.
The evaluation should provide the following functionality for users:
An administrator updates a sensitive configuration value. The administrator requires the change to take effect at runtime without republishing the agent.

A builder needs to confirm a configuration value but must not be able to modify it from Copilot Studio.
An administrator updates a non-secret configuration value and requires the updated value to be used by the published agent at runtime.
You need to create a test set-based evaluation that supports consistent test runs for each scenario.
What should you configure for each scenario? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Test configuration

Scenario	Configuration
Update a sensitive configuration value and the change takes effect at runtime without republishing the agent.	Only in production Only after publishing Secret environment variable
Confirm a configuration value but not modify it from Copilot Studio.	Publishing settings Conversation volume Managed environment variable
Update a non-secret configuration value and the updated value needs to be used by the published agent at runtime.	String environment variable Live chat history Channel rollout status

Answer:

Test configuration

Scenario	Configuration
Update a sensitive configuration value and the change takes effect at runtime without republishing the agent.	Only in production Only after publishing **Secret environment variable**
Confirm a configuration value but not modify it from Copilot Studio.	Publishing settings Conversation volume **Managed environment variable**
Update a non-secret configuration value and the updated value needs to be used by the published agent at runtime.	**String environment variable** Live chat history Channel rollout status

Explanation:

Sensitive configurations (like API keys or tokens) are securely backed by Azure Key Vault using a **Secret environment variable**. Since environment variables are referenced dynamically, updating the value updates the bot's runtime behavior immediately without needing a full republish.

When a solution is imported into a target environment (like UAT or Production) as **Managed**, the assets

become read-only to prevent citizen developers from modifying them from the canvas. You can confirm or view a **Managed environment variable** value, but you cannot edit its core definition inside Copilot Studio.

For standard non-sensitive text configurations (like a base URL or a contact email), a **String environment variable** is used. Any live update made to this value will be dynamically pulled by the published agent at runtime.

Question: 74 CertyIQ

HOTSPOT -
A team is preparing to evaluate an agent in Copilot Studio before expanding access to additional users.
The team must select an evaluation method that meets the following requirements:
Use a fixed set of prepared interactions.
Determine responses against a predefined baseline.
Support consistent comparison across repeated test runs.
You need to determine the evaluation method that satisfies the evaluation requirements.
Which evaluation method should you select for each requirement? To answer, select the appropriate option in the answer area.
NOTE: Each correct selection is worth one point.

Evaluation methods

Requirement	Evaluation method
Use a fixed set of prepared interactions.	Use a curated prompt list. Use only live conversations. Expand access to increase traffic.
Establish a baseline for determining responses.	Confirm publishing status as success. Rely on general impressions of quality. Compare against predefined expected outcomes.
Support repeatable comparisons across runs.	Collect user feedback after rollout. Use a consistent scoring evaluation matrix. Change channel settings before publishing.

Answer:

Evaluation methods

Requirement	Evaluation method
Use a fixed set of prepared interactions.	Use a curated prompt list. Use only live conversations. Expand access to increase traffic.
Establish a baseline for determining responses.	Confirm publishing status as success. Rely on general impressions of quality. Compare against predefined expected outcomes.
Support repeatable comparisons across runs.	Collect user feedback after rollout. Use a consistent scoring evaluation matrix. Change channel settings before publishing.

Explanation:

Use a curated prompt list. (Stops random variables from creeping into the test).

Establishing a Baseline → Compare against predefined expected outcomes. (Provides the ground truth comparison target).

Support Repeatable Comparisons → Use a consistent scoring evaluation matrix. (Ensures mathematical and qualitative uniformity across separate developer test runs).

Question: 75 CertyIQ

A solution containing an agent in Copilot Studio is moved across environments. The agent relies on configuration values that are different between environments and are managed as environment variables.
You must ensure the following:
Configuration values are managed through environment variables that supports ALM.
Runtime behavior reflects updates to environment variable values.
You need to implement environment-variable usage that supports ALM and ensures correct runtime behavior.
Which three actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Replace environment variables with hard-coded values inside the agent so that runtime behavior is always consistent across environments.

B. Allow builders to change environment variable values directly in Copilot Studio during topic authoring to speed deployment.

C. Use environment variables as read-only in Copilot Studio and change their values in the appropriate administrative authoring experience.

D. Store external reference values as environment variables so the agent solution can move between environments while keeping the agent definition the same.

E. Assume that all environment variable updates take effect immediately at runtime for all variable types without republishing.

F. After an administrator updates a non-secret environment variable, republish any agents that use the variable so the change is effective at runtime.

Answer: CDF

Explanation:

C. Use environment variables as read-only in Copilot Studio and change their values in the appropriate administrative authoring experience.

D. Store external reference values as environment variables so the agent solution can move between environments while keeping the agent definition the same.

F. After an administrator updates a non-secret environment variable, republish any agents that use the variable so the change is effective at runtime.

Action C (Platform Boundary): Environment variables are natively **read-only** within the Microsoft Copilot Studio authoring canvas. Their actual configuration and value management must occur externally inside the **Power Apps solution explorer** or admin centers.
Action D (ALM Enablement): External reference values (such as API endpoints, base URLs, or tenant specific configurations) change when transitioning from a Development landscape into Test or Production. Moving these values into environment variables prevents hardcoding, allowing the parent solution wrapper to

transport safely without modifying core topic code definitions.
Action F (Runtime Behavior Integration): In Copilot Studio, standard non-secret environment variable parameters are captured into the agent metadata layer during the compilation phase. If an administrator makes changes to these variables, the **agent must be republished** for those newly updated values to take effect at production runtime.

Question: 76 CertyIQ

HOTSPOT -
An organization uses separate development and production environments for agents in Copilot Studio.
You create an agent in the development environment.
The agent must be deployed to production by using an ALM process.
You must configure the solution strategy to meet the following requirements:
Developers continue modifying the agent in development.
The production deployment prevents direct modification.
The agent is exported from development and imported into production in a deployment-ready state.
You need to configure solutions to support deployment and lifecycle management of the agents.
Which solution type should you use for each requirement? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Requirement	Solution type
Allow ongoing development.	Default solution Managed solution Unmanaged solution Standalone agent outside a solution
Prevent modification in production.	Default solution Managed solution Temporary solution Unmanaged solution
Export from development and import into production in deployment-ready state.	Default solution Managed solution Unmanaged solution Standalone agent outside a solution

Answer:

Requirement	Solution type
Allow ongoing development.	Default solution Managed solution **Unmanaged solution** Standalone agent outside a solution
Prevent modification in production.	Default solution **Managed solution** Temporary solution Unmanaged solution
Export from development and import into production in deployment-ready state.	Default solution **Managed solution** Unmanaged solution Standalone agent outside a solution

Explanation:

1. Allow ongoing development → Unmanaged solution

Reasoning: Unmanaged solutions are designated for active authoring, testing, and modification. They act as the primary containers for developers or makers to create new elements or configure existing configurations. There are no locking constraints placed on components within an unmanaged solution in a development sandbox environment.

2. Prevent modification in production → Managed solution

Reasoning: When a solution is installed as a **managed solution** in an environment (such as a production or user acceptance testing environment), it locks the underlying assets. This enforces stability and safety protocols by preventing end-users or administrators from modifying components directly in the production environment, which ensures consistency with source control.

3. Export from development and import into production in deployment-ready state → Managed solution

Reasoning: To transport customizations in a finalized, secure, and deployment-ready state, you export your unmanaged development solution as a **managed** package. This package is then imported into the target production environment. The managed container structure allows for clean version tracking, simplified application upgrades, and complete asset removal upon uninstallation.

Question: 77

CertyIQ

A team is standardizing ALM for agents in Copilot Studio.
The team requires a custom solution that includes agents. The solution must support ALM scenarios, such as

managing and transporting agent assets across environments and pipeline deployments.
You need to create an ALM-ready custom solution in Copilot Studio.
What should you do?

A. Publish the agent to a channel and use that channel configuration as the ALM container.
B. Create a new custom solution from the Solutions area.
C. Export the custom solution as a managed solution and import it into production.
D. Export the agent directly and then import it into another environment.

Answer: B

Explanation:

B. Create a new custom solution from the Solutions area.

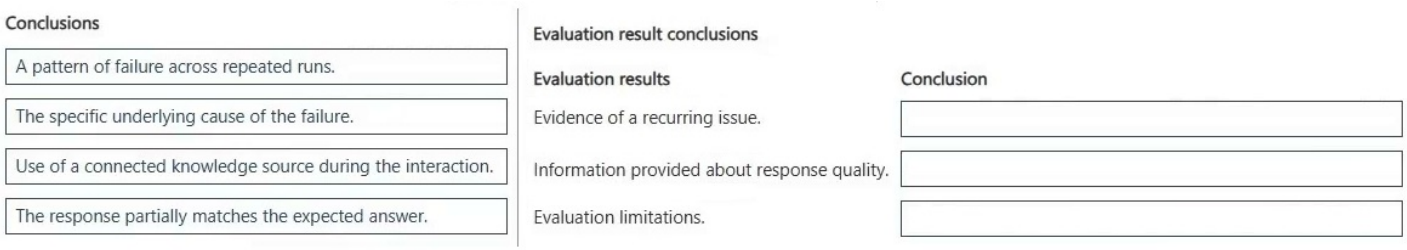
Creating the Container: Before you can build an ALM-ready agent, you must first create an unmanaged custom solution within the **Solutions** navigation area of the Copilot Studio Portal (or Power Apps Maker portal).

Managing Dependencies: Working within a dedicated custom solution allows you to group your agent with its respective components (such as topics, environment variables, flows, and connection references), ensuring that the entire bundle can be cleanly moved across environments.

Question: 78

CertyIQ

DRAG DROP -
You run the same fixed test set three times in Copilot Studio.
During evaluation, you observe the following:
The same interaction fails in all three runs.
Score values range from 0.58 to 0.61.
The reasoning states that the response partially matches the expected answer.
The knowledge source that is used is internal documentation.
No tools are invoked.
You need to determine which conclusions are supported based on the evaluation results.
Which conclusions should you make? To answer, move the appropriate conclusions to the correct evaluation results. You may use each conclusion once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:

Conclusions

A pattern of failure across repeated runs.

The specific underlying cause of the failure.

Use of a connected knowledge source during the interaction.

The response partially matches the expected answer.

Evaluation result conclusions

Evaluation results	Conclusion
Evidence of a recurring issue.	A pattern of failure across repeated runs.
Information provided about response quality.	The response partially matches the expected answer.
Evaluation limitations.	The specific underlying cause of the failure.

Explanation:

Evidence of a recurring issue. → A pattern of failure across repeated runs.

Information provided about response quality. → The response partially matches the expected answer.

Evaluation limitations. → The specific underlying cause of the failure.

Question: 79 CertyIQ

DRAG DROP -
A team wants to automate solution deployment between environments. Deployments must follow a controlled, repeatable process.
The team needs to implement a pipeline-based deployment process that enforces the correct dependency order.
You need to implement a deployment pipeline and determine the correct sequence of actions.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

Pipeline implementation sequence	Answer Area
Assign environments to pipeline stages.	1
Trigger a pipeline run.	2
Enable pipelines for the tenant.	3
Review deployment results.	4

Answer:

Pipeline implementation sequence	Answer Area
Assign environments to pipeline stages.	1 Enable pipelines for the tenant.
Trigger a pipeline run.	2 Assign environments to pipeline stages.
Enable pipelines for the tenant.	3 Trigger a pipeline run.
Review deployment results.	4 Review deployment results.

Explanation:

Enable pipelines for the tenant.

Assign environments to pipeline stages.

Trigger a pipeline run.

Review deployment results.

Question: 80

CertyIQ

A company needs an agent in Copilot Studio to complete tasks by interacting with a legacy desktop application that does not use APIs. The builder needs to validate that UI automation can run in the supported execution context and needs a way to inspect how the automation behaves during testing conversations.
To meet the business needs, the solution must meet the following requirements:
Enable the agent to perform UI automation against the legacy application.
Inspect execution details for the automation during conversations.
You need to configure the agent based on the requirements.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Configure a custom connector for the desktop app so the agent can automate the UI through authenticated calls.

B. Add a Power Platform Desktop flow to control the desktop app.

C. Access the remote session within the testing pane to monitor the real-time execution of the agent's desktop interactions.

D. Publish the agent so the automation runs in the supported environment automatically.

E. Enable the Computer use capability.

Answer: CE

Explanation:

Testing Pane & Remote Session (C): When testing a computer-use-enabled agent inside the Copilot Studio testing pane, makers can open a live remote desktop session overlay. This lets you inspect execution details and watch the agent navigate, take screenshots, and execute mouse/keyboard events in real time during a test conversation.

Computer use capability (E): The Computer Use tool in Copilot Studio is built exactly for this purpose. It allows AI agents to visually navigate desktop/web apps, type fields, and click buttons on a remote machine using natural language descriptions, providing resilient UI automation against legacy systems where traditional APIs do not exist.

Question: 81

CertyIQ

DRAG DROP -
A development team is reusing agents in Copilot Studio across development, test, and production environments.
You need to ensure the agents meet the following requirements:
Reusable components must support governed promotion across environments.
Environment-specific configuration must not be hard-coded.
Deployments must support automated, multi-stage approvals.
You need to configure the agents based on the business requirements.
Which configurations should you use for each requirement? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

- Use an ALM pipeline.
- Export as a component collection.
- Use connection references and environment variables.

ALM configuration options

Requirement	Configuration
Share authored topics and prompts across environments.	
Move environment-specific configurations including connections and secrets.	
Automate multi-stage deployments with approvals.	

Answer:

Configurations

- Use an ALM pipeline.
- Export as a component collection.
- Use connection references and environment variables.

ALM configuration options

Requirement	Configuration
Share authored topics and prompts across environments.	Export as a component collection.
Move environment-specific configurations including connections and secrets.	Use connection references and environment variables.
Automate multi-stage deployments with approvals.	Use an ALM pipeline.

Explanation:

Share authored topics and prompts across environments:

Export as a component collection.

Move environment-specific configurations including connections and secrets:

Use connection references and environment variables.

Automate multi-stage deployments with approvals:

Use an ALM pipeline.

Question: 82

CertyIQ

An agent calls a flow that intermittently produces unexpected results during execution.
The agent behaves inconsistently.
You need to investigate agent flow executions to diagnose the issue.
Which two actions should you use? Each correct answer presents a complete solution.
NOTE: Each correct selection is worth one point.

A. Inspect failed run details.
B. Update the flow trigger definition.
C. Check the Error column in the Activity tab.
D. Review flow analytics summary.
E. Review flow run history.
F. Increase the connector timeout.

Answer: AE

Explanation:

Inspect failed run details (A): Clicking into a specific failed execution from the history allows you to view the visual execution path. You can look at each individual action step, inspect the exact inputs/outputs, and read the specific error message or status code produced by the failure to diagnose the issue.

Review flow run history (E): This provides a complete chronological list of all recent executions (Succeeded, Failed, or Running). It serves as the starting point for seeing exactly when and how often an execution has failed.

Question: 83 CertyIQ

You create a test set in Copilot Studio to evaluate an agent that answers policy questions and retrieves data from connected knowledge sources.
You run the evaluation and review the following result for one test case:
Expected response: Digital products are non-refundable after download.
Actual response: Digital products cannot be refunded once downloaded.
Result: Pass -
Score: 0.92 -
Reasoning: The actual response semantically matches the expected response.
Knowledge sources used: RefundPolicy_KB
Tools invoked: None -
You need to determine whether additional corrective action is required based on the evaluation results.
Which two actions should you perform? Each correct answer presents a complete solution.
NOTE: Each correct selection is worth one point.

A. Investigate external tool configuration for this test case.
B. Rerun the evaluation because the score is below 1.00.
C. Accept the result because semantic matching is sufficient for a passing evaluation.
D. Confirm that the connected knowledge source is functioning as expected.
E. Modify the test case because the response is not an exact match.

Answer: CD

Explanation:

C. Accept the result because semantic matching is sufficient for a passing evaluation.
D. Confirm that the connected knowledge source is functioning as expected.

Accepting semantic matches (C): In conversational AI and agent testing, responses generated via Large Language Models (LLMs) rarely provide an exact 100% word-for-word match to the test script. Since the evaluation checks for information quality, if a response is a partial or semantic match that fulfills the intent, it is considered a passing evaluation and requires no corrective action.
Verifying knowledge sources (D): Because agent interactions heavily rely on grounding data, ensuring that any connected knowledge sources (such as SharePoint sites, websites, or internal repositories) are online and properly indexed is a critical diagnostic step to ensure responses remain accurate across repeated test runs.

Question: 84 CertyIQ

HOTSPOT -
A team deploys Copilot Studio solutions by using Power Platform Pipelines.
You must configure a deployment strategy that meets the following requirements:
Ensure that the solutions deploy in dev, test, and production environments in order.
Identify missing connection references or environment variables before the deployment process begins.
Manage all pipeline stages and security from a single, unified location for all linked environments.
You need to configure the pipeline to meet the requirements.
What should you configure for each requirement? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Pipeline deployment controls

Requirement	Configuration
Solutions deploy in dev, test and production environments in order.	Sequential stage progression Direct target environment selection Environment-specific deployment packages
Identify missing connection references or environment variables before deployment.	Deployment input validation Managed solution deployment Deployment configuration override
Manage pipeline stages and security from a single, unified location for linked environments.	Environment-level customization Centralized pipeline configuration Builder-defined deployment settings

Answer:

Pipeline deployment controls

Requirement	Configuration
Solutions deploy in dev, test and production environments in order.	**Sequential stage progression** Direct target environment selection Environment-specific deployment packages
Identify missing connection references or environment variables before deployment.	**Deployment input validation** Managed solution deployment Deployment configuration override
Manage pipeline stages and security from a single, unified location for linked environments.	Environment-level customization **Centralized pipeline configuration** Builder-defined deployment settings

Explanation:

1. Sequential stage progression

Why it fits: In an ideal Application Lifecycle Management (ALM) flow, updates must move systematically from **Development \(\rightarrow \) Test \(\rightarrow \) Production** without bypassing any intermediate environments. Implementing a sequence restriction ensures that a solution cannot accidentally be pushed into a live Production environment before successfully passing the verification stages in the Test environment.

2. Deployment input validation

Why it fits: Before code or solutions are applied to a target environment, the system performs a series of **pre-flight checks**. This automated verification evaluates if all dependency structures, connection references, and required environment variables have valid values assigned. Identifying missing parameters before execution prevents broken configurations or runtime errors post-deployment.

3. Centralized pipeline configuration

Why it fits: In enterprise platforms (like Microsoft Power Platform), a single dedicated environment acts as the **Pipeline Host**. Administering pipeline definitions, stage connections, access controls, and deployment permissions from this one centralized location ensures consistent governance across all linked developer and target environments.

Question: 85 CertyIQ

DRAG DROP -
A team is preparing a test set to evaluate an agent in Copilot Studio before expanding access to additional users. The test set must support consistent evaluation across repeated runs and focus on the agent's most important behaviors.
You need to create a test set that enables repeatable and meaningful evaluation.
In which order should you perform the actions? To answer, move all actions from the list of actions to the answer area and arrange them in the correct order.

Agent performance evaluation sequence	Answer Area	
Define the expected outcomes and success criteria for each interaction.	1	
Author representative user prompts that trigger the identified intents.	2	
Identify the high-value conversational intents the agent must handle.	3	
Baseline the test set to ensure deterministic evaluation across repeated runs.	4	

Answer:

Agent performance evaluation sequence
Define the expected outcomes and success criteria for each interaction.
Author representative user prompts that trigger the identified intents.
Identify the high-value conversational intents the agent must handle.
Baseline the test set to ensure deterministic evaluation across repeated runs.

Answer Area	
1	Identify the high-value conversational intents the agent must handle.
2	Author representative user prompts that trigger the identified intents.
3	Define the expected outcomes and success criteria for each interaction.
4	Baseline the test set to ensure deterministic evaluation across repeated runs.

Explanation:

Identify the high-value conversational intents the agent must handle.

Author representative user prompts that trigger the identified intents.

Define the expected outcomes and success criteria for each interaction.

Baseline the test set to ensure deterministic evaluation across repeated runs.

1. **Identify Intents**: You must first establish the scope of what the agent needs to do by choosing the key, high-value tasks and user intentions it is responsible for handling.
2. **Author Prompts**: Once the targeted intents are established, you write realistic user phrases and inputs to explicitly trigger those specific intents during evaluation.
3. **Define Success Criteria**: For every test prompt created, you must specify what a correct or successful response looks like (e.g., specific entities extracted or correct topic triggered) to grade the agent fairly.
4. **Baseline the Test Set**: Finally, lock in the test dataset to establish a repeatable baseline. This ensures you can run identical benchmarks later and accurately measure if future agent updates improve or degrade performance.

Question: 86

CertyIQ

DRAG DROP -
A team is preparing to assess an agent in Copilot Studio before expanding access to additional users.
The team requires an evaluation that provides controlled, repeatable, and consistently measured test runs.
The evaluation must:
Be triggered directly by the team to control when testing occurs.
Determine success based on clearly established acceptance criteria.
Rely on a predefined source that ensures consistency across repeated runs.
You need to configure the evaluation to meet the requirements.
What should you configure? To answer, move the appropriate configurations to the correct evaluation requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations
Execute the evaluation on demand.
Expand rollout to additional channels.
Enable analytics to monitor agent usage trends.
Compare responses against defined expectations.
Use a curated prompt list as the interaction source.

Pipeline configurations

Evaluation Requirement	Configuration
Triggered directly by the team to control when testing occurs.	
Determine success based on clearly established acceptance criteria.	
Rely on a predefined source that ensures consistency across repeated runs.	

Answer:

Configurations
Execute the evaluation on demand.
Expand rollout to additional channels.
Enable analytics to monitor agent usage trends.
Compare responses against defined expectations.
Use a curated prompt list as the interaction source.

Pipeline configurations

Evaluation Requirement	Configuration
Triggered directly by the team to control when testing occurs.	Execute the evaluation on demand.
Determine success based on clearly established acceptance criteria.	Compare responses against defined expectations.
Rely on a predefined source that ensures consistency across repeated runs.	Use a curated prompt list as the interaction source.

Explanation:

On-Demand Execution: Allows developers or QA teams to manually trigger an evaluation run at specific milestones (like after a major code change) instead of relying on a scheduled or automated trigger.
Comparing Against Expectations: Directly assesses whether the agent's output satisfies the pre-configured success criteria or ground truth.
Curated Prompt List: Serves as a standardized test harness or benchmark dataset, ensuring the agent is always evaluated against the exact same inputs to prevent testing variability.

Question: 87

CertyIQ

HOTSPOT -
You run multiple evaluation tests for an agent in Copilot Studio before expanding user access.
You observe the following about the evaluation results:
Each evaluation case is reported as meeting or missing the expected response.
Some evaluation cases fail repeatedly across several runs.
Each run displays the expected response and the response generated by the agent.
You need to interpret what the evaluation results reveal.
What should you conclude based on each interpretation? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Evaluation interpretation

Interpretation	Supported conclusion
Outcome of individual evaluation cases	Pass or fail status / Failure configuration / Production readiness
Behavior observed across repeated runs	Prompt revisions / Consistent outcomes / Intermittent outcomes
Details presented in evaluation output	Deployment approval / Root cause of failures / Expected and generated responses

Answer:

Evaluation interpretation

Interpretation	Supported conclusion
Outcome of individual evaluation cases	**Pass or fail status**
Behavior observed across repeated runs	**Consistent outcomes**
Details presented in evaluation output	**Expected and generated responses**

Explanation:

1. Pass or fail status

Why it fits: When analyzing a single, distinct test case or user interaction, the most direct result is a binary evaluation — determining whether that specific case met the criteria (**Pass**) or missed it (**Fail**).

2. Consistent outcomes

Why it fits: Running the exact same evaluation suite multiple times helps identify system reliability. Observing stable, identical behavior across these repeated iterations indicates **consistent outcomes**, confirming the system is deterministic and predictable.

3. Expected and generated responses

Why it fits: To understand why a test run behaved the way it did, the granular test output reports the raw text generated by the agent side-by-side with the target ground truth. This raw comparison of **expected vs. generated responses** allows developers to manually audit differences.

Question: 88 CertyIQ

DRAG DROP -
A company uses an agent in Copilot Studio in production and requires centralized monitoring for operational visibility and troubleshooting.
The team decides to use Application Insights as the monitoring solution.
An agent activity is not included in the monitoring data.
You need to enable monitoring the agent by using Application Insights.
Which four actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

Agent monitoring
Enable monitoring for the agent.
Publish the agent so the monitoring configuration takes effect.
Configure which monitoring destination the agent uses.
Create or identify the Application Insights destination for telemetry.
Configure the agent feature that generates grounded answers.
Configure the agent feature that selects a model from a catalog.

Answer Area	
1	
2	
3	
4	

Answer:

Agent monitoring
Enable monitoring for the agent.
Publish the agent so the monitoring configuration takes effect.
Configure which monitoring destination the agent uses.
Create or identify the Application Insights destination for telemetry.
Configure the agent feature that generates grounded answers.
Configure the agent feature that selects a model from a catalog.

Answer Area	
1	Create or identify the Application Insights destination for telemetry.
2	Configure which monitoring destination the agent uses.
3	Enable monitoring for the agent.
4	Publish the agent so the monitoring configuration takes effect.

Explanation:

1. **Create or identify the Application Insights destination for telemetry.**
2. **Configure which monitoring destination the agent uses.**
3. **Enable monitoring for the agent.**
4. **Publish the agent so the monitoring configuration takes effect.**

Step 1 (Infrastructure Setup): You must begin in the Azure portal by creating an Application Insights resource (or locating an existing one) to obtain the connection string that will host and index the telemetry log streams.

Step 2 (Destination Mapping): In Copilot Studio, navigate to **Settings** > **Advanced** > **Application Insights** and paste the copied Azure connection string. This connects the conversational agent interface to your cloud storage environment.

Step 3 (Configuration): Once the destination is bound, toggle on or choose the specific logging behaviors (such as recording user activity properties, messages, or custom topic events) to activate telemetry capturing.

Step 4 (Deployment): Telemetry configurations remain local and inactive until a build is compiled. You must **Publish the agent** to push these telemetry endpoints into production so that conversation logs begin routing live to Azure Monitor.

Question: 89 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A team is implementing ALM for agents in Copilot Studio across multiple environments.
The team requires a consistent way to manage and transport agent assets.
You need to recommend a solution that supports creation and use of ALM scenarios for agents in Copilot Studio.
Solution: To export, import, and manage agents between environments, you need to create and use a custom solution.
Does the solution meet the goal?

A. Yes

B. No

Answer: A

Explanation:

A. Yes

In the Power Platform ecosystem, a **solution** (specifically a custom solution created by the development team) is the official and required ALM container used to package, manage, transport, and deploy assets — including Microsoft Copilot Studio agents, topics, and environment variables — across multiple environments (such as Development, Test, and Production).

Question: 90 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A team is implementing ALM for agents in Copilot Studio across multiple environments.
The team requires a consistent way to manage and transport agent assets.
You need to recommend a solution that supports creation and use of ALM scenarios for agents in Copilot Studio.
Solution: Publishing an agent to a channel is the required ALM container for transporting agents between environments.
Does the solution meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

Publishing an agent simply makes it live and accessible to end-users on a specific channel (such as Microsoft Teams or a public website) within its **current** environment. It is not a deployment tool.

The required ALM container for transporting Microsoft Copilot Studio agents between distinct environments (e.g., from Development to Test or Production) is a **Power Platform Solution**.

Question: 91 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.
A team is implementing ALM for agents in Copilot Studio across multiple environments.
The team requires a consistent way to manage and transport agent assets.
You need to recommend a solution that supports creation and use of ALM scenarios for agents in Copilot Studio.
Solution: Builders can change environment variable values directly in Copilot Studio to complete ALM

configuration without administrative tools.
Does the solution meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

While builders can use **environment variables** within Copilot Studio to change configuration data (such as API endpoints) between different environments, doing so does **not** provide a mechanism to actually manage and transport the copilot assets across environments.

To properly implement Application Lifecycle Management (ALM) for Copilot Studio agents, you must group the agent components into a **Power Platform Solution** and export/import that solution package (via Pipelines, DevOps, or administrative tools) between your dev, test, and production environments. Modifying variable values directly inside Copilot Studio without these transport tools does not satisfy the requirements of a consistent asset deployment strategy.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

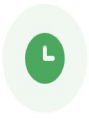
You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers

Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com

Free Updates

Lifetime Free Updates!

Total: **91 Questions**

Link: https://certyiq.com/papers/microsoft/ab-620